Premier essaie de génération d'images

In [86]:
```
# Suppression des warnings FutureWarning
import warnings  # Gère les warnings
warnings.filterwarnings("ignore", category=FutureWarning)

# Librairies générales
import pandas as pd  # Manipulation de données tabulaires
import numpy as np  # Calcul scientifique et manipulation de matrices
import os  # Interaction avec le système de fichiers
from os.path import isfile, join  # Fonctions de gestion des chemins de fichiers
import sys  # Interaction avec l'interpréteur Python
import random  # Génération de nombres aléatoiresr
import zipfile  # Manipulation d'archives zip
from scipy.stats import norm, entropy # Statistiques sur les données

# Librairie d'affichage
import matplotlib.pyplot as plt  # Visualisation de données
from matplotlib.offsetbox import OffsetImage, AnnotationBbox  # Visualisation de d
from PIL import Image, ImageDraw  # Traitement d'images avec Pillow

# Bibliothèque scikit-learn
from sklearn.decomposition import PCA  # Réduction de dimension
from sklearn.manifold import TSNE  # Visualisation de données en dimension réduite

# TensorFlow et Keras
import tensorflow as tf  # API principale de TensorFlow
from tensorflow.keras.losses import mse  # Perte par erreur quadratique moyenne
from keras import layers, models, optimizers  # Création et gestion des modèles
from tensorflow.keras.preprocessing.image import ImageDataGenerator # Générateur
from keras.preprocessing.image import img_to_array, load_img, array_to_img  # Prép
from keras.callbacks import ModelCheckpoint, EarlyStopping  # Gestion des callback
from keras.layers import (Input, Conv2D, Flatten, Dense, Conv2DTranspose, Reshape,
                          Lambda, Activation, BatchNormalization, LeakyReLU, Drop
                          MaxPooling2D, UpSampling2D)  # Différentes couches pour
from keras.optimizers import Adam  # Optimiseur Adam
from keras.models import Model, load_model  # Définition et chargement des modèles
from keras.layers import LeakyReLU  # Activation LeakyReLU
import tensorflow.keras.backend as K # Import des opérations bas niveau de Keras
```

Définition des répertoires pour stocker les modéles appris

In [87]:
```
#!rm Data_humanface_cat_without_caption_1000.zip
# Définition du répertoire cible
model_dir = "./Data_humanface_cat_without_caption_1000/"

# Création du répertoire s'il n'existe pas
os.makedirs(model_dir, exist_ok=True)

zip_file = "Data_humanface_cat_without_caption_1000.zip"

!wget https://www.lirmm.fr/~poncelet/Ressources/Data_humanface_cat_without_caption


# Extraction du fichier ZIP
with zipfile.ZipFile(zip_file, "r") as zip_ref:
        zip_ref.extractall(model_dir)

# Suppression du fichier ZIP après extraction pour économiser de l'espace
os.remove(zip_file)
```

```
--2026-05-24 23:00:23--  https://www.lirmm.fr/~poncelet/Ressources/Data_humanface_
cat_without_caption_1000.zip
Resolving www.lirmm.fr (www.lirmm.fr)... 193.49.104.251
Connecting to www.lirmm.fr (www.lirmm.fr)|193.49.104.251|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 42306490 (40M) [application/zip]
Saving to: ‘Data_humanface_cat_without_caption_1000.zip’

Data_humanface_cat_ 100%[===================>]  40.35M  1.63MB/s    in 48s

2026-05-24 23:01:11 (866 KB/s) - ‘Data_humanface_cat_without_caption_1000.zip’ sav
ed [42306490/42306490]
```

another way to download data idk which is better? # Nom local de l'archive et URL source zip_file = "Data_humanface_cat_without_caption_.zip" zip_url = "https://www.lirmm.fr/~poncelet/Ressources/Data_humanface_cat_without_caption_400.zip" # Répertoire cible après extraction src_dir = "images" image_dir = os.path.join(src_dir, "man") # Vérifie si le dossier de destination contient des images def humans_already_present(): return os.path.exists(image_dir) and any(f.endswith(".jpg") for f in os.listdir(image_dir)) # Téléchargement et extraction seulement si besoin if not humans_already_present(): print("Aucune image trouvée dans Cat. Téléchargement et extraction en cours...") # Télécharger si l'archive n'est pas déjà présente if not os.path.exists(zip_file): print(f"Téléchargement de {zip_file}...") !wget -O {zip_file} {zip_url} else: print(f"{zip_file} déjà présent, téléchargement ignoré.") # Extraction dans le répertoire courant with zipfile.ZipFile(zip_file, "r") as zip_ref: for member in zip_ref.namelist(): if not member.startswith("__MACOSX"): zip_ref.extract(member, ".") print("Archive extraite dans le répertoire courant.") # Suppression de l'archive après extraction os.remove(zip_file) else: print("Le dossier Humans/train existe déjà et contient des images. Téléchargement ignoré.")

In [88]:
```
##affichage des images
```

In [89]:
```
# Répertoire contenant les images
src_dir = "Data_humanface_cat_without_caption_1000/images"
image_dir = os.path.join(src_dir, "cat")

files = [f for f in os.listdir(image_dir) if f.endswith(".jpg")]

print("Nombre d'images dans", image_dir, ":", len(files))

if files:
    with Image.open(os.path.join(image_dir, files[0])) as img:
        shape = img.size[::-1] + (3,)  # (height, width, 3)
        print("Format (shape) des images :", shape)


# Tirage de 5 images aléatoires
random_files = random.sample(files, 5)

# Affichage
plt.figure(figsize=(8, 2))
for i, fname in enumerate(random_files):
    img_path = os.path.join(image_dir, fname)
    img = Image.open(img_path)

    plt.subplot(1, 5, i + 1)
    plt.imshow(img)
    plt.title(fname, fontsize=8)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Nombre d'images dans Data_humanface_cat_without_caption_1000/images/cat : 1000
Format (shape) des images : (256, 256, 3)
```

2. Importer le jeu de données

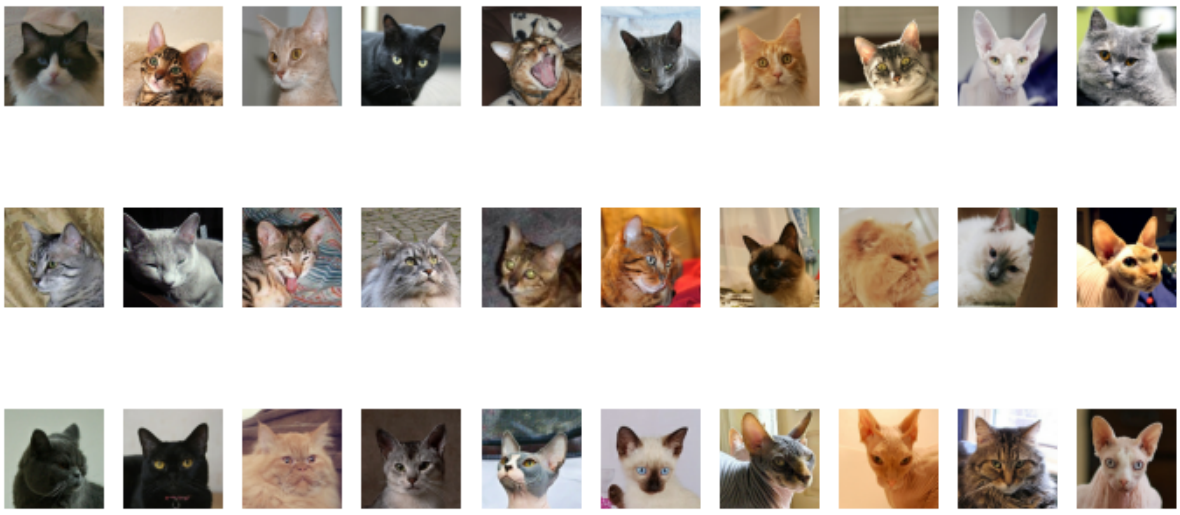
In [90]:

```
image_size = (128, 128, 3)  # Dimensions des images (hauteur, largeur, canaux)
batch_size = 512             # Taille des lots
latent_dim = 200             # Dimension du vecteur latent pour l'autoencodeur

# Génération des données normalisées (valeurs entre 0 et 1)
data_flow = ImageDataGenerator(rescale=1./255).flow_from_directory(
    directory=src_dir,
    classes=["cat"],  # Le sous-dossier contenant toutes les images
    target_size=image_size[:2],  # Redimension à 128x128
    batch_size=batch_size,
    shuffle=True,
    seed=42,
    class_mode='input'  # Autoencodeur : entrée = sortie
)
```

```
Found 1000 images belonging to 1 classes.
```

In [91]:

```
# Obtention du premier batch d'images générées
data_batch = next(data_flow)

# Affichage des 30 premières images du batch
import matplotlib.pyplot as plt

fig, axes = plt.subplots(3, 10, figsize=(12, 6))
for i, ax in enumerate(axes.flatten()):
    ax.imshow(data_batch[0][i])
    ax.axis('off')
plt.show()
```



1/ Définition de l'encodeur et décodeur

In [92]:

```
def build_encoder(input_dim, output_dim):
    tf.keras.backend.clear_session()
```

```
    encoder_input = Input(shape=input_dim, name='encoder_input')
    x = encoder_input

    x = Conv2D(32, kernel_size=3, strides=2, padding='same',
               name='encoder_conv_1')(x)
    x = LeakyReLU(name='encoder_act_1')(x)

    x = Conv2D(64, kernel_size=3, strides=2, padding='same',
               name='encoder_conv_2')(x)
    x = LeakyReLU(name='encoder_act_2')(x)

    x = Conv2D(64, kernel_size=3, strides=2, padding='same',
               name='encoder_conv_3')(x)
    x = LeakyReLU(name='encoder_act_3')(x)

    x = Conv2D(64, kernel_size=3, strides=2, padding='same',
               name='encoder_conv_4')(x)
    x = LeakyReLU(name='encoder_act_4')(x)

    shape_before_flattening = tf.keras.backend.int_shape(x)[1:]
    x = Flatten()(x)
    encoder_output = Dense(output_dim, name='encoder_output')(x)

    encoder = Model(encoder_input, encoder_output, name='encoder')
    return encoder_input, encoder_output, shape_before_flattening, encoder

def build_decoder(input_dim, shape_before_flattening):
    # Entrée du décodeur (vecteur latent)
    decoder_input = Input(shape=(input_dim,), name='decoder_input')

    # Projeter dans un tenseur de même forme qu'avant le Flatten de l'encodeur
    x = Dense(np.prod(shape_before_flattening))(decoder_input)
    x = Reshape(shape_before_flattening)(x)

    # Couches de déconvolution (effet miroir de l'encodeur)
    x = Conv2DTranspose(64, kernel_size=3, strides=2, padding='same',
                        name='decoder_conv_1')(x)
    x = LeakyReLU(name='decoder_act_1')(x)

    x = Conv2DTranspose(64, kernel_size=3, strides=2, padding='same',
                        name='decoder_conv_2')(x)
    x = LeakyReLU(name='decoder_act_2')(x)

    x = Conv2DTranspose(32, kernel_size=3, strides=2, padding='same',
                        name='decoder_conv_3')(x)
    x = LeakyReLU(name='decoder_act_3')(x)

    x = Conv2DTranspose(3, kernel_size=3, strides=2, padding='same',
                        name='decoder_conv_4')(x)
    decoder_output = Activation('sigmoid', name='decoder_output')(x)

    decoder = Model(decoder_input, decoder_output, name='decoder')
    return decoder_input, decoder_output, decoder
```

In [93]:

```
image_size = (128, 128, 3)  # Dimensions des images (hauteur, largeur, canaux)
batch_size = 512            # Taille des lots
latent_dim = 200            # Dimension du vecteur latent pour l'autoencodeur

# Création de l'encodeur avec les dimensions d'entrée et de sortie spécifiées
encoder_input, encoder_output, shape_before_flattening, encoder = build_encoder(
    input_dim=image_size,
    output_dim=latent_dim
```

```
)

# Création du décodeur en utilisant la dimension de l'espace latent
# et le shape  avant le flattening
decoder_input, decoder_output, decoder = build_decoder(
    input_dim=latent_dim,
    shape_before_flattening=shape_before_flattening
)
```

Partie autoencodeur

In [94]:

```
# L'entrée du modèle autoencodeur est celle de l'encodeur
autoencoder_input = encoder_input

# La sortie est obtenue en appliquant le décodeur à la sortie de l'encodeur
autoencoder_output = decoder(encoder_output)

# Création du modèle autoencodeur complet (encodeur + décodeur)
autoencoder = Model(autoencoder_input, autoencoder_output, name='autoencoder')

# Affichage de la structure du modèle
autoencoder.summary()
```

Model: "autoencoder"

Layer (type)	Output Shape	Param #
encoder_input (InputLayer)	(None, 128, 128, 3)	0
encoder_conv_1 (Conv2D)	(None, 64, 64, 32)	896
encoder_act_1 (LeakyReLU)	(None, 64, 64, 32)	0
encoder_conv_2 (Conv2D)	(None, 32, 32, 64)	18,496
encoder_act_2 (LeakyReLU)	(None, 32, 32, 64)	0
encoder_conv_3 (Conv2D)	(None, 16, 16, 64)	36,928
encoder_act_3 (LeakyReLU)	(None, 16, 16, 64)	0
encoder_conv_4 (Conv2D)	(None, 8, 8, 64)	36,928
encoder_act_4 (LeakyReLU)	(None, 8, 8, 64)	0
flatten (Flatten)	(None, 4096)	0
encoder_output (Dense)	(None, 200)	819,400
decoder (Functional)	(None, 128, 128, 3)	916,483

Total params: 1,829,131 (6.98 MB)

Trainable params: 1,829,131 (6.98 MB)

Non-trainable params: 0 (0.00 B)

In [95]:

```
# Génération des données normalisées (valeurs entre 0 et 1)
data_flow = ImageDataGenerator(rescale=1./255).flow_from_directory(
    directory=src_dir,
```

```
    classes=['cat'],  # Le sous-dossier contenant toutes les images
    target_size=image_size[:2],  # Redimension à 128x128
    batch_size=batch_size,
    shuffle=True,
    seed=42,
    class_mode='input'  # Autoencodeur : entrée = sortie
)
```

```
Found 1000 images belonging to 1 classes.
```

Espace latent

In [96]:

```
def visualize_latent_space(encoder_model, images, num_images=100,
                           image_size=(28, 28), zoom=0.6):
    """
    Visualise la répartition de l'espace latent en utilisant t-SNE
    et affiche des vignettes d'images.

    Paramètres :
    -----------
    encoder_model : Model
        Le modèle d'encodeur (autoencodeur ou VAE) à utiliser pour obtenir
        les vecteurs latents.
    images : array-like
        Batch d'images d'entrée pour la visualisation.
    num_images : int, optional
        Nombre d'images à utiliser pour la visualisation (par défaut 100).
    image_size : tuple, optional
        Taille des vignettes d'image à afficher (par défaut (28, 28)).
    zoom : float, optional
        Facteur de zoom pour les vignettes d'image (par défaut 0.6).
    """
    # Limite le nombre d'images à `num_images`
    images = images[:num_images]

    # Génère les vecteurs latents avec l'encodeur
    latent_vectors = encoder_model.predict(images)
    if isinstance(latent_vectors, list):
        latent_vectors = latent_vectors[0]

    # Aplatit les vecteurs latents s'ils ont plus de 2 dimensions
    if latent_vectors.ndim > 2:
        latent_vectors = latent_vectors.reshape((latent_vectors.shape[0], -1))

    # Réduction de dimension avec t-SNE pour projeter en 2D
    tsne = TSNE(n_components=2, random_state=42)
    latent_2d = tsne.fit_transform(latent_vectors)

    plt.figure(figsize=(15, 12))
    ax = plt.gca()

    # Affichage des images dans l'espace t-SNE avec AnnotationBbox
    for i, (x, y) in enumerate(latent_2d):
        # Conversion de l'image en vignette
        thumbnail = array_to_img(images[i])
        thumbnail = thumbnail.resize(image_size)
        imagebox = OffsetImage(thumbnail, zoom=zoom)
        ab = AnnotationBbox(imagebox, (x, y), frameon=False)
        ax.add_artist(ab)

    all_x, all_y = latent_2d[:, 0], latent_2d[:, 1]
    ax.set_xlim(all_x.min() - 5, all_x.max() + 5)
    ax.set_ylim(all_y.min() - 5, all_y.max() + 5)
```

```
    plt.title("Répartition des images dans l'espace latent")
    plt.xlabel("Dimension 1")
    plt.ylabel("Dimension 2")

    ax.grid(False)
    plt.show()
```

In [97]:

```
#my guess is we need this before?
# Définition des hyperparamètres
epochs = 150
model_name = "catmodeltrain.keras"
model_path = os.path.join(model_dir, model_name)

# Compilation du modèle autoencodeur
autoencoder.compile(
    optimizer='adam',
    loss='binary_crossentropy'
)

# Générateur principal avec normalisation  et séparation train/validation
train_gen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

# Générateur pour les données d'entraînement
train_flow = train_gen.flow_from_directory(
    directory=src_dir,
    classes=['cat'],  # le sous-dossier contenant toutes les images
    target_size=image_size[:2],
    batch_size=batch_size,
    shuffle=True,
    seed=42,
    class_mode='input',
    subset='training'
)

# Générateur pour les données de validation
val_flow = train_gen.flow_from_directory(
    directory=src_dir,
    classes=['cat'],
    target_size=image_size[:2],
    batch_size=batch_size,
    shuffle=False,
    seed=42,
    class_mode='input',
    subset='validation'
)


# Entraînement du modèle autoencodeur
#autoencoder = train_autoencoder_from_generator(
   # model=autoencoder,
    #train_gen=train_flow,
    #val_gen=val_flow,
    #model_path=model_path,
    #epochs=epochs,
    #patience=5
#)
```

```
Found 800 images belonging to 1 classes.
Found 200 images belonging to 1 classes.
```

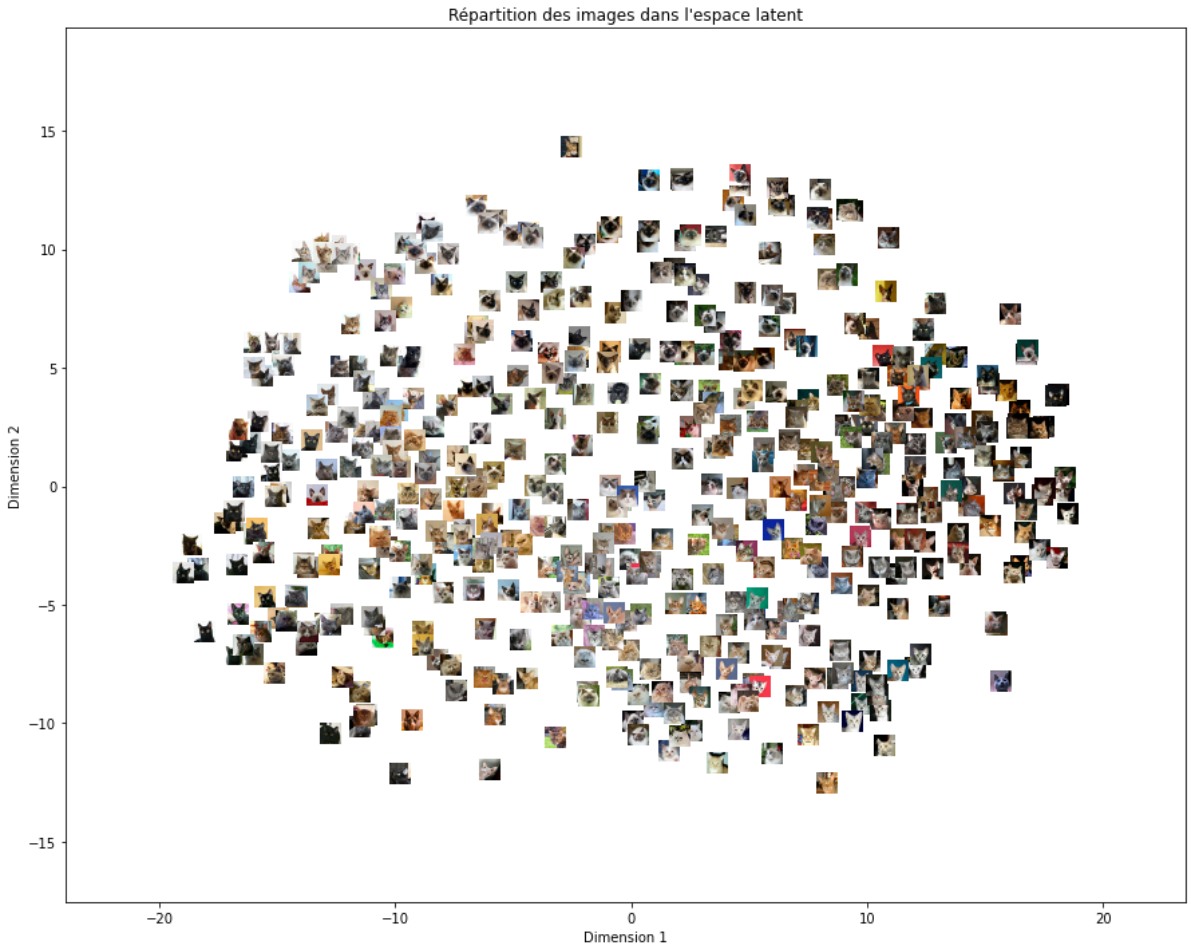
In [98]:

```
# Chemin vers le modèle entraîné
model_path = os.path.join(model_dir, "catmodeltrain.keras")

# Chargement du modèle
autoencoder = load_model(model_path)

# Récupération du premier lot d'images
example_batch = next(data_flow)

# Extraction des images
example_batch = example_batch[0]
```

In [99]:

```
# Visualisation de l'espace latent avec les vignettes
visualize_latent_space(
    encoder_model=encoder,
    images=example_batch,
    num_images=batch_size,
    image_size=(32, 32),
    zoom=0.5
)
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step
```



Transformation auto-encodeur en VAEOn modifie l'encodeur : On veut un encodeur qui à chaque image d'entrée retourne une distribution dans l'espace latent avec 2 paramétres: moyenne et log variance 1. on inclut la couche de perte

In [100…

```
image_size = (128, 128, 3)  # Dimensions des images (hauteur, largeur, canaux)
batch_size = 512            # Taille des lots
latent_dim = 200            # Dimension du vecteur latent pour l'autoencodeur
```

In [101…

```
import tensorflow as tf
# Fonction de sampling avec reparameterization trick
@tf.keras.utils.register_keras_serializable()
def sampling(args):
    mean_mu, log_var = args
    epsilon = K.random_normal(shape=K.shape(mean_mu), mean=0., stddev=1.)
    return mean_mu + K.exp(log_var / 2) * epsilon

# Couche personnalisée pour le calcul de la perte KL
@tf.keras.utils.register_keras_serializable()
class KLLossLayer(tf.keras.layers.Layer):
    def call(self, inputs):
        mean_mu, log_var = inputs
        kl_loss = -0.5 * tf.reduce_sum(1 + log_var - tf.square(mean_mu) - tf.exp(l
        self.add_loss(kl_loss)
        return kl_loss

# VAE Encoder
def build_vae_encoder(input_dim, output_dim):
    tf.keras.backend.clear_session()

    vae_encoder_input = Input(shape=input_dim, name='vae_encoder_input')
    x = vae_encoder_input

    x = Conv2D(32, 3, strides=2, padding='same', name='vae_encoder_conv_1')(x)
    x = LeakyReLU(name='vae_encoder_act_1')(x)

    x = Conv2D(64, 3, strides=2, padding='same', name='vae_encoder_conv_2')(x)
    x = LeakyReLU(name='vae_encoder_act_2')(x)

    x = Conv2D(64, 3, strides=2, padding='same', name='vae_encoder_conv_3')(x)
    x = LeakyReLU(name='vae_encoder_act_3')(x)

    x = Conv2D(64, 3, strides=2, padding='same', name='vae_encoder_conv_4')(x)
    x = LeakyReLU(name='vae_encoder_act_4')(x)

    shape_before_flattening = tf.keras.backend.int_shape(x)[1:]
    x = Flatten(name='vae_encoder_flatten')(x)

    mean_mu = Dense(output_dim, name='mu')(x)
    log_var = Dense(output_dim, name='log_var')(x)

    vae_encoder_output = Lambda(sampling, output_shape=(output_dim,),
                                name='vae_encoder_output')([mean_mu, log_var])

    _ = KLLossLayer(name='kl_loss')([mean_mu, log_var])

    vae_encoder = Model(vae_encoder_input, vae_encoder_output, name='vae_encoder')
    return vae_encoder_input, vae_encoder_output, mean_mu, log_var, shape_before_f
```

In [102…

```
# Construction de l'encodeur VAE
vae_encoder_input, vae_encoder_output, mean_mu, log_var, shape_before_flattening,
    input_dim=image_size,
    output_dim=latent_dim
)

# Affichage de la structure de l'encodeur
vae_encoder.summary()
```

Model: "vae_encoder"

Layer (type)	Output Shape	Param #	Connected to
vae_encoder_input (InputLayer)	(None, 128, 128, 3)	0	-
vae_encoder_conv_1 (Conv2D)	(None, 64, 64, 32)	896	vae_encoder_inpu…
vae_encoder_act_1 (LeakyReLU)	(None, 64, 64, 32)	0	vae_encoder_conv…
vae_encoder_conv_2 (Conv2D)	(None, 32, 32, 64)	18,496	vae_encoder_act_…
vae_encoder_act_2 (LeakyReLU)	(None, 32, 32, 64)	0	vae_encoder_conv…
vae_encoder_conv_3 (Conv2D)	(None, 16, 16, 64)	36,928	vae_encoder_act_…
vae_encoder_act_3 (LeakyReLU)	(None, 16, 16, 64)	0	vae_encoder_conv…
vae_encoder_conv_4 (Conv2D)	(None, 8, 8, 64)	36,928	vae_encoder_act_…
vae_encoder_act_4 (LeakyReLU)	(None, 8, 8, 64)	0	vae_encoder_conv…
vae_encoder_flatten (Flatten)	(None, 4096)	0	vae_encoder_act_…
mu (Dense)	(None, 200)	819,400	vae_encoder_flat…
log_var (Dense)	(None, 200)	819,400	vae_encoder_flat…
vae_encoder_output (Lambda)	(None, 200)	0	mu[0][0], log_var[0][0]

Total params: 1,732,048 (6.61 MB)

Trainable params: 1,732,048 (6.61 MB)

même décodeur donc pas besoin de redefinir

In [103…

```
# Construction du décodeur VAE
decoder_input, decoder_output, decoder = build_decoder(
    input_dim=latent_dim,
    shape_before_flattening=shape_before_flattening
)

# Affichage de la structure du décodeur
decoder.summary()
```

Model: "decoder"

Layer (type)	Output Shape	Param #
decoder_input (InputLayer)	(None, 200)	0
dense (Dense)	(None, 4096)	823,296
reshape (Reshape)	(None, 8, 8, 64)	0
decoder_conv_1 (Conv2DTranspose)	(None, 16, 16, 64)	36,928
decoder_act_1 (LeakyReLU)	(None, 16, 16, 64)	0
decoder_conv_2 (Conv2DTranspose)	(None, 32, 32, 64)	36,928
decoder_act_2 (LeakyReLU)	(None, 32, 32, 64)	0
decoder_conv_3 (Conv2DTranspose)	(None, 64, 64, 32)	18,464
decoder_act_3 (LeakyReLU)	(None, 64, 64, 32)	0
decoder_conv_4 (Conv2DTranspose)	(None, 128, 128, 3)	867
decoder_output (Activation)	(None, 128, 128, 3)	0

Total params: 916,483 (3.50 MB)

Trainable params: 916,483 (3.50 MB)

En combinant l'encodeur variationnel et le décodeur pour former notre VAE :

In [104…

```
# Initialisation de l'encodeur VAE
vae_encoder_input, vae_encoder_output, mean_mu, log_var, shape_before_flattening,
    input_dim=image_size,
    output_dim=latent_dim
)

# Initialisation du décodeur (identique à l'autoencodeur classique)
decoder_input, decoder_output, decoder = build_decoder(
    input_dim=latent_dim,
    shape_before_flattening=shape_before_flattening
)

# Construction du VAE complet
vae_output = decoder(vae_encoder_output)
vae = Model(inputs=vae_encoder_input, outputs=vae_output, name="VAE")

# Compilation du modèle avec la reconstruction loss (la KL est intégrée
# dans l'encodeur)
B = 1.0  # facteur de pondération de la reconstruction

def r_loss(y_true, y_pred):
    return K.mean(K.square(y_true - y_pred), axis=[1, 2, 3]) # MSE pixel à pixel

def total_loss(y_true, y_pred):
    return B * r_loss(y_true, y_pred) # KL-divergence ajoutée via .add_loss()
```

```
vae.compile(optimizer=Adam(), loss=total_loss)

# Affichage du résumé du modèle
vae.summary()
```

Model: "VAE"

Layer (type)	Output Shape	Param #	Connected to
vae_encoder_input (InputLayer)	(None, 128, 128, 3)	0	-
vae_encoder_conv_1 (Conv2D)	(None, 64, 64, 32)	896	vae_encoder_inpu…
vae_encoder_act_1 (LeakyReLU)	(None, 64, 64, 32)	0	vae_encoder_conv…
vae_encoder_conv_2 (Conv2D)	(None, 32, 32, 64)	18,496	vae_encoder_act_…
vae_encoder_act_2 (LeakyReLU)	(None, 32, 32, 64)	0	vae_encoder_conv…
vae_encoder_conv_3 (Conv2D)	(None, 16, 16, 64)	36,928	vae_encoder_act_…
vae_encoder_act_3 (LeakyReLU)	(None, 16, 16, 64)	0	vae_encoder_conv…
vae_encoder_conv_4 (Conv2D)	(None, 8, 8, 64)	36,928	vae_encoder_act_…
vae_encoder_act_4 (LeakyReLU)	(None, 8, 8, 64)	0	vae_encoder_conv…
vae_encoder_flatten (Flatten)	(None, 4096)	0	vae_encoder_act_…
mu (Dense)	(None, 200)	819,400	vae_encoder_flat…
log_var (Dense)	(None, 200)	819,400	vae_encoder_flat…
vae_encoder_output (Lambda)	(None, 200)	0	mu[0][0], log_var[0][0]
decoder (Functional)	(None, 128, 128, 3)	916,483	vae_encoder_outp…

Total params: 2,648,531 (10.10 MB)

Trainable params: 2,648,531 (10.10 MB)

Non-trainable params: 0 (0.00 B)

définition de la fonction train adapté au VAE

In [105…

```
# Génération des données normalisées (valeurs entre 0 et 1)
data_flow = ImageDataGenerator(rescale=1./255).flow_from_directory(
    directory=src_dir,
    classes=['cat'],  # Le sous-dossier contenant toutes les images
```

```
    target_size=image_size[:2],  # Redimension à 128x128
    batch_size=batch_size,
    shuffle=True,
    seed=42,
    class_mode='input'  # Autoencodeur : entrée = sortie
)
```

```
Found 1000 images belonging to 1 classes.
```

In [106…

```
def train_vae_from_generator(model, train_gen, val_gen,
                             model_path, epochs=100, patience=5):
    """
    Entraîne un modèle VAE avec des générateurs Keras.

    Args:
        model       : modèle VAE compilé
        train_gen   : générateur pour l'entraînement
        val_gen     : générateur pour la validation
        model_path  : chemin de sauvegarde du meilleur modèle
        epochs      : nombre maximal d'époques
        patience    : patience pour l'arrêt anticipé

    Returns:
        Le modèle avec les meilleurs poids restaurés.
    """
    callbacks = [
        EarlyStopping(monitor='val_loss', patience=patience,
                      restore_best_weights=True, verbose=1),
        ModelCheckpoint(filepath=model_path,
                        monitor='val_loss',
                        save_best_only=True,
                        save_weights_only=False,
                        verbose=1)
    ]

    history = model.fit(
        train_gen,
        validation_data=val_gen,
        epochs=epochs,
        callbacks=callbacks,
        verbose=1
    )

    return model
```

In [107…

```
# Définition des hyperparamètres
epochs = 150
model_name = "catGeneratingVAE.keras"
model_path = os.path.join(model_dir, model_name)

# Générateur principal avec normalisation et séparation train/validation
train_gen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

# Générateur pour les données d'entraînement
train_flow = train_gen.flow_from_directory(
    directory=src_dir,
    classes=['cat'],  # le sous-dossier contenant toutes les images
    target_size=image_size[:2],
    batch_size=batch_size,
```

```
    shuffle=True,
    seed=42,
    class_mode='input',
    subset='training'
)

# Générateur pour les données de validation
val_flow = train_gen.flow_from_directory(
    directory=src_dir,
    classes=['cat'],
    target_size=image_size[:2],
    batch_size=batch_size,
    shuffle=False,
    seed=42,
    class_mode='input',
    subset='validation'
)

# Entraînement du modèle VAE
vae = train_vae_from_generator(
    model=vae,
    train_gen=train_flow,
    val_gen=val_flow,
    model_path=model_path,
    epochs=epochs,
    patience=5
)
```

```
Found 800 images belonging to 1 classes.
Found 200 images belonging to 1 classes.
Epoch 1/150
E0000 00:00:1779656539.759205   15148 meta_optimizer.cc:967] remapper failed: INVA
LID_ARGUMENT: Mutation::Apply error: fanout 'StatefulPartitionedCall/gradient_tap
e/VAE_1/vae_encoder_act_3_1/LeakyRelu/LeakyReluGrad' exist for missing node 'State
fulPartitionedCall/VAE_1/vae_encoder_conv_3_1/BiasAdd'.
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0736
Epoch 1: val_loss improved from None to 0.07463, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 14s 3s/step - loss: 0.0733 - val_loss: 0.0746
Epoch 2/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0735
Epoch 2: val_loss improved from 0.07463 to 0.07447, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 13s 2s/step - loss: 0.0731 - val_loss: 0.0745
Epoch 3/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 913ms/step - loss: 0.0724
Epoch 3: val_loss improved from 0.07447 to 0.07427, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0729 - val_loss: 0.0743
Epoch 4/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 795ms/step - loss: 0.0724
Epoch 4: val_loss improved from 0.07427 to 0.07405, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0727 - val_loss: 0.0741
Epoch 5/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0721
Epoch 5: val_loss improved from 0.07405 to 0.07376, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0723 - val_loss: 0.0738
Epoch 6/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0733
Epoch 6: val_loss improved from 0.07376 to 0.07342, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0719 - val_loss: 0.0734
Epoch 7/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 815ms/step - loss: 0.0716
Epoch 7: val_loss improved from 0.07342 to 0.07305, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0714 - val_loss: 0.0731
Epoch 8/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0718
Epoch 8: val_loss improved from 0.07305 to 0.07282, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0708 - val_loss: 0.0728
Epoch 9/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0713
Epoch 9: val_loss improved from 0.07282 to 0.07272, saving model to ./Data_humanfa
ce_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0704 - val_loss: 0.0727
Epoch 10/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 858ms/step - loss: 0.0710
Epoch 10: val_loss improved from 0.07272 to 0.07239, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0700 - val_loss: 0.0724
Epoch 11/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0698
Epoch 11: val_loss improved from 0.07239 to 0.07075, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0693 - val_loss: 0.0707
Epoch 12/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0672
Epoch 12: val_loss improved from 0.07075 to 0.06681, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0675 - val_loss: 0.0668
Epoch 13/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0639
Epoch 13: val_loss improved from 0.06681 to 0.06417, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0629 - val_loss: 0.0642
```

```
Epoch 14/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0609
Epoch 14: val_loss improved from 0.06417 to 0.05756, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0597 - val_loss: 0.0576
Epoch 15/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 835ms/step - loss: 0.0555
Epoch 15: val_loss improved from 0.05756 to 0.05472, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0554 - val_loss: 0.0547
Epoch 16/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 818ms/step - loss: 0.0534
Epoch 16: val_loss improved from 0.05472 to 0.05305, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0531 - val_loss: 0.0530
Epoch 17/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 811ms/step - loss: 0.0522
Epoch 17: val_loss improved from 0.05305 to 0.05096, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0514 - val_loss: 0.0510
Epoch 18/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 801ms/step - loss: 0.0494
Epoch 18: val_loss improved from 0.05096 to 0.04857, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0491 - val_loss: 0.0486
Epoch 19/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0465
Epoch 19: val_loss improved from 0.04857 to 0.04597, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0462 - val_loss: 0.0460
Epoch 20/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0443
Epoch 20: val_loss improved from 0.04597 to 0.04311, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0434 - val_loss: 0.0431
Epoch 21/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 887ms/step - loss: 0.0419
Epoch 21: val_loss did not improve from 0.04311
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - loss: 0.0418 - val_loss: 0.0475
Epoch 22/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 185ms/step - loss: 0.0462
Epoch 22: val_loss did not improve from 0.04311
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 852ms/step - loss: 0.0442 - val_loss: 0.0475
Epoch 23/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 770ms/step - loss: 0.0444
Epoch 23: val_loss improved from 0.04311 to 0.03956, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - loss: 0.0427 - val_loss: 0.0396
Epoch 24/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 778ms/step - loss: 0.0386
Epoch 24: val_loss improved from 0.03956 to 0.03781, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 2s/step - loss: 0.0383 - val_loss: 0.0378
Epoch 25/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0359
Epoch 25: val_loss improved from 0.03781 to 0.03744, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0363 - val_loss: 0.0374
Epoch 26/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0355
Epoch 26: val_loss improved from 0.03744 to 0.03520, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 3s/step - loss: 0.0351 - val_loss: 0.0352
Epoch 27/150
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 831ms/step - loss: 0.0340
Epoch 27: val_loss improved from 0.03520 to 0.03390, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0337 - val_loss: 0.0339
Epoch 28/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 782ms/step - loss: 0.0325
Epoch 28: val_loss improved from 0.03390 to 0.03185, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - loss: 0.0324 - val_loss: 0.0318
Epoch 29/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0305
Epoch 29: val_loss improved from 0.03185 to 0.03066, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0312 - val_loss: 0.0307
Epoch 30/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0298
Epoch 30: val_loss improved from 0.03066 to 0.02988, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0303 - val_loss: 0.0299
Epoch 31/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0292
Epoch 31: val_loss improved from 0.02988 to 0.02873, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0294 - val_loss: 0.0287
Epoch 32/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 859ms/step - loss: 0.0282
Epoch 32: val_loss did not improve from 0.02873
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0278 - val_loss: 0.0290
Epoch 33/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 832ms/step - loss: 0.0280
Epoch 33: val_loss improved from 0.02873 to 0.02761, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0276 - val_loss: 0.0276
Epoch 34/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0266
Epoch 34: val_loss improved from 0.02761 to 0.02727, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 3s/step - loss: 0.0265 - val_loss: 0.0273
Epoch 35/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 783ms/step - loss: 0.0256
Epoch 35: val_loss improved from 0.02727 to 0.02627, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0258 - val_loss: 0.0263
Epoch 36/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 769ms/step - loss: 0.0253
Epoch 36: val_loss improved from 0.02627 to 0.02611, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - loss: 0.0252 - val_loss: 0.0261
Epoch 37/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0250
Epoch 37: val_loss improved from 0.02611 to 0.02510, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0245 - val_loss: 0.0251
Epoch 38/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0241
Epoch 38: val_loss improved from 0.02510 to 0.02453, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0240 - val_loss: 0.0245
Epoch 39/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0229
Epoch 39: val_loss improved from 0.02453 to 0.02399, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0233 - val_loss: 0.0240
Epoch 40/150
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 864ms/step - loss: 0.0229
Epoch 40: val_loss improved from 0.02399 to 0.02352, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0231 - val_loss: 0.0235
Epoch 41/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0222
Epoch 41: val_loss improved from 0.02352 to 0.02313, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0225 - val_loss: 0.0231
Epoch 42/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 3s/step - loss: 0.0222
Epoch 42: val_loss improved from 0.02313 to 0.02289, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 7s 4s/step - loss: 0.0220 - val_loss: 0.0229
Epoch 43/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 52s/step - loss: 0.0218
Epoch 43: val_loss improved from 0.02289 to 0.02234, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 57s 55s/step - loss: 0.0218 - val_loss: 0.0223
Epoch 44/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 867ms/step - loss: 0.0211
Epoch 44: val_loss did not improve from 0.02234
2/2 ━━━━━━━━━━━━━━━━━━━━ 5s 1s/step - loss: 0.0213 - val_loss: 0.0224
Epoch 45/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0214
Epoch 45: val_loss improved from 0.02234 to 0.02165, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0210 - val_loss: 0.0216
Epoch 46/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 693ms/step - loss: 0.0205
Epoch 46: val_loss improved from 0.02165 to 0.02159, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0206 - val_loss: 0.0216
Epoch 47/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 592ms/step - loss: 0.0205
Epoch 47: val_loss improved from 0.02159 to 0.02108, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 999ms/step - loss: 0.0205 - val_loss: 0.0211
Epoch 48/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0198
Epoch 48: val_loss did not improve from 0.02108
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 1s/step - loss: 0.0200 - val_loss: 0.0211
Epoch 49/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 623ms/step - loss: 0.0201
Epoch 49: val_loss did not improve from 0.02108
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 956ms/step - loss: 0.0201 - val_loss: 0.0218
Epoch 50/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0204
Epoch 50: val_loss improved from 0.02108 to 0.02038, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0200 - val_loss: 0.0204
Epoch 51/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 622ms/step - loss: 0.0192
Epoch 51: val_loss improved from 0.02038 to 0.02019, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0193 - val_loss: 0.0202
Epoch 52/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0188
Epoch 52: val_loss improved from 0.02019 to 0.01986, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - loss: 0.0191 - val_loss: 0.0199
Epoch 53/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0186
Epoch 53: val_loss improved from 0.01986 to 0.01976, saving model to ./Data_humanf
```

```
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0187 - val_loss: 0.0198
Epoch 54/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 685ms/step - loss: 0.0185
Epoch 54: val_loss improved from 0.01976 to 0.01966, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0186 - val_loss: 0.0197
Epoch 55/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 581ms/step - loss: 0.0186
Epoch 55: val_loss did not improve from 0.01966
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 860ms/step - loss: 0.0185 - val_loss: 0.0200
Epoch 56/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 585ms/step - loss: 0.0187
Epoch 56: val_loss did not improve from 0.01966
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 908ms/step - loss: 0.0189 - val_loss: 0.0207
Epoch 57/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0193
Epoch 57: val_loss improved from 0.01966 to 0.01906, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 1s/step - loss: 0.0190 - val_loss: 0.0191
Epoch 58/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0181
Epoch 58: val_loss did not improve from 0.01906
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 1s/step - loss: 0.0180 - val_loss: 0.0192
Epoch 59/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0181
Epoch 59: val_loss improved from 0.01906 to 0.01865, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0180 - val_loss: 0.0187
Epoch 60/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0178
Epoch 60: val_loss did not improve from 0.01865
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 1s/step - loss: 0.0176 - val_loss: 0.0188
Epoch 61/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 600ms/step - loss: 0.0176
Epoch 61: val_loss improved from 0.01865 to 0.01838, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0175 - val_loss: 0.0184
Epoch 62/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 615ms/step - loss: 0.0171
Epoch 62: val_loss improved from 0.01838 to 0.01833, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0173 - val_loss: 0.0183
Epoch 63/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0171
Epoch 63: val_loss improved from 0.01833 to 0.01821, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0171 - val_loss: 0.0182
Epoch 64/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0168
Epoch 64: val_loss improved from 0.01821 to 0.01797, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - loss: 0.0170 - val_loss: 0.0180
Epoch 65/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 620ms/step - loss: 0.0168
Epoch 65: val_loss improved from 0.01797 to 0.01782, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0168 - val_loss: 0.0178
Epoch 66/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 667ms/step - loss: 0.0168
Epoch 66: val_loss improved from 0.01782 to 0.01770, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0167 - val_loss: 0.0177
Epoch 67/150
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0165
Epoch 67: val_loss improved from 0.01770 to 0.01763, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0166 - val_loss: 0.0176
Epoch 68/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0165
Epoch 68: val_loss did not improve from 0.01763
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 1s/step - loss: 0.0165 - val_loss: 0.0177
Epoch 69/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0167
Epoch 69: val_loss did not improve from 0.01763
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - loss: 0.0166 - val_loss: 0.0182
Epoch 70/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0173
Epoch 70: val_loss did not improve from 0.01763
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - loss: 0.0171 - val_loss: 0.0185
Epoch 71/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 664ms/step - loss: 0.0171
Epoch 71: val_loss improved from 0.01763 to 0.01719, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0169 - val_loss: 0.0172
Epoch 72/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0165
Epoch 72: val_loss did not improve from 0.01719
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 1s/step - loss: 0.0163 - val_loss: 0.0174
Epoch 73/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0165
Epoch 73: val_loss improved from 0.01719 to 0.01719, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - loss: 0.0161 - val_loss: 0.0172
Epoch 74/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0165
Epoch 74: val_loss did not improve from 0.01719
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0162 - val_loss: 0.0173
Epoch 75/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0162
Epoch 75: val_loss improved from 0.01719 to 0.01696, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - loss: 0.0159 - val_loss: 0.0170
Epoch 76/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 671ms/step - loss: 0.0159
Epoch 76: val_loss improved from 0.01696 to 0.01688, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0159 - val_loss: 0.0169
Epoch 77/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0153
Epoch 77: val_loss improved from 0.01688 to 0.01685, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0156 - val_loss: 0.0168
Epoch 78/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0156
Epoch 78: val_loss improved from 0.01685 to 0.01664, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0156 - val_loss: 0.0166
Epoch 79/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0158
Epoch 79: val_loss improved from 0.01664 to 0.01658, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0154 - val_loss: 0.0166
Epoch 80/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0156
Epoch 80: val_loss improved from 0.01658 to 0.01644, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - loss: 0.0154 - val_loss: 0.0164
```

```
Epoch 81/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 755ms/step - loss: 0.0154
Epoch 81: val_loss improved from 0.01644 to 0.01640, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0152 - val_loss: 0.0164
Epoch 82/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0155
Epoch 82: val_loss did not improve from 0.01640
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0152 - val_loss: 0.0164
Epoch 83/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 624ms/step - loss: 0.0154
Epoch 83: val_loss improved from 0.01640 to 0.01630, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0152 - val_loss: 0.0163
Epoch 84/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0152
Epoch 84: val_loss improved from 0.01630 to 0.01613, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0150 - val_loss: 0.0161
Epoch 85/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0152
Epoch 85: val_loss did not improve from 0.01613
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0149 - val_loss: 0.0161
Epoch 86/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0149
Epoch 86: val_loss improved from 0.01613 to 0.01612, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0149 - val_loss: 0.0161
Epoch 87/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 815ms/step - loss: 0.0148
Epoch 87: val_loss improved from 0.01612 to 0.01606, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0149 - val_loss: 0.0161
Epoch 88/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 703ms/step - loss: 0.0150
Epoch 88: val_loss did not improve from 0.01606
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0148 - val_loss: 0.0162
Epoch 89/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 747ms/step - loss: 0.0150
Epoch 89: val_loss did not improve from 0.01606
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0151 - val_loss: 0.0165
Epoch 90/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 664ms/step - loss: 0.0154
Epoch 90: val_loss improved from 0.01606 to 0.01593, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0154 - val_loss: 0.0159
Epoch 91/150
1/2 ━━━━━━━━━╺━━━━━━━━━━ 1s 2s/step - loss: 0.0145
Epoch 91: val_loss did not improve from 0.01593
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 389ms/step - loss: 0.0147 - val_loss: 0.0161
Epoch 92/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0150
Epoch 92: val_loss improved from 0.01593 to 0.01590, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0148 - val_loss: 0.0159
Epoch 93/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0147
Epoch 93: val_loss improved from 0.01590 to 0.01565, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0145 - val_loss: 0.0157
Epoch 94/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0141
Epoch 94: val_loss did not improve from 0.01565
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0145 - val_loss: 0.0158
```

```
Epoch 95/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0146
Epoch 95: val_loss improved from 0.01565 to 0.01550, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0145 - val_loss: 0.0155
Epoch 96/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0144
Epoch 96: val_loss did not improve from 0.01550
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0142 - val_loss: 0.0156
Epoch 97/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0144
Epoch 97: val_loss did not improve from 0.01550
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0143 - val_loss: 0.0155
Epoch 98/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0143
Epoch 98: val_loss improved from 0.01550 to 0.01534, saving model to ./Data_humanf
ace_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0141 - val_loss: 0.0153
Epoch 99/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 722ms/step - loss: 0.0141
Epoch 99: val_loss did not improve from 0.01534
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0141 - val_loss: 0.0154
Epoch 100/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0139
Epoch 100: val_loss improved from 0.01534 to 0.01532, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0141 - val_loss: 0.0153
Epoch 101/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0139
Epoch 101: val_loss improved from 0.01532 to 0.01518, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0139 - val_loss: 0.0152
Epoch 102/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0137
Epoch 102: val_loss did not improve from 0.01518
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 1s/step - loss: 0.0139 - val_loss: 0.0154
Epoch 103/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 756ms/step - loss: 0.0142
Epoch 103: val_loss did not improve from 0.01518
2/2 ━━━━━━━━━━━━━━━━━━━━ 6s 1s/step - loss: 0.0141 - val_loss: 0.0159
Epoch 104/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 934ms/step - loss: 0.0144
Epoch 104: val_loss did not improve from 0.01518
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0144 - val_loss: 0.0153
Epoch 105/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0137
Epoch 105: val_loss improved from 0.01518 to 0.01507, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0138 - val_loss: 0.0151
Epoch 106/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0137
Epoch 106: val_loss did not improve from 0.01507
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0139 - val_loss: 0.0153
Epoch 107/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0138
Epoch 107: val_loss improved from 0.01507 to 0.01490, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 2s/step - loss: 0.0138 - val_loss: 0.0149
Epoch 108/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 960ms/step - loss: 0.0135
Epoch 108: val_loss did not improve from 0.01490
2/2 ━━━━━━━━━━━━━━━━━━━━ 4s 1s/step - loss: 0.0135 - val_loss: 0.0152
Epoch 109/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 849ms/step - loss: 0.0139
```

```
Epoch 109: val_loss improved from 0.01490 to 0.01489, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0138 - val_loss: 0.0149
Epoch 110/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0135
Epoch 110: val_loss did not improve from 0.01489
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0134 - val_loss: 0.0151
Epoch 111/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 654ms/step - loss: 0.0137
Epoch 111: val_loss did not improve from 0.01489
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 332ms/step - loss: 0.0138 - val_loss: 0.0150
Epoch 112/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0135
Epoch 112: val_loss improved from 0.01489 to 0.01486, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0134 - val_loss: 0.0149
Epoch 113/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 665ms/step - loss: 0.0133
Epoch 113: val_loss did not improve from 0.01486
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0135 - val_loss: 0.0153
Epoch 114/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 741ms/step - loss: 0.0139
Epoch 114: val_loss improved from 0.01486 to 0.01461, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0137 - val_loss: 0.0146
Epoch 115/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - loss: 0.0135
Epoch 115: val_loss did not improve from 0.01461
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0133 - val_loss: 0.0148
Epoch 116/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 772ms/step - loss: 0.0133
Epoch 116: val_loss improved from 0.01461 to 0.01448, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0132 - val_loss: 0.0145
Epoch 117/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 689ms/step - loss: 0.0130
Epoch 117: val_loss did not improve from 0.01448
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0130 - val_loss: 0.0145
Epoch 118/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0133
Epoch 118: val_loss improved from 0.01448 to 0.01428, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0130 - val_loss: 0.0143
Epoch 119/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 694ms/step - loss: 0.0128
Epoch 119: val_loss improved from 0.01428 to 0.01424, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0128 - val_loss: 0.0142
Epoch 120/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 641ms/step - loss: 0.0125
Epoch 120: val_loss improved from 0.01424 to 0.01414, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0127 - val_loss: 0.0141
Epoch 121/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0130
Epoch 121: val_loss improved from 0.01414 to 0.01394, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0126 - val_loss: 0.0139
Epoch 122/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 730ms/step - loss: 0.0123
Epoch 122: val_loss improved from 0.01394 to 0.01394, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0124 - val_loss: 0.0139
Epoch 123/150
```

```
1/2 ━━━━━━━━━━━━━━━━━━━━ 1s 2s/step - loss: 0.0120
Epoch 123: val_loss improved from 0.01394 to 0.01370, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 184ms/step - loss: 0.0124 - val_loss: 0.0137
Epoch 124/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0122
Epoch 124: val_loss improved from 0.01370 to 0.01359, saving model to ./Data_human
face_cat_without_caption_1000/catGeneratingVAE.keras
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0121 - val_loss: 0.0136
Epoch 125/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0123
Epoch 125: val_loss did not improve from 0.01359
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0122 - val_loss: 0.0141
Epoch 126/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0126
Epoch 126: val_loss did not improve from 0.01359
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0124 - val_loss: 0.0140
Epoch 127/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - loss: 0.0124
Epoch 127: val_loss did not improve from 0.01359
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 2s/step - loss: 0.0121 - val_loss: 0.0141
Epoch 128/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 670ms/step - loss: 0.0125
Epoch 128: val_loss did not improve from 0.01359
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0125 - val_loss: 0.0153
Epoch 129/150
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 803ms/step - loss: 0.0139
Epoch 129: val_loss did not improve from 0.01359
2/2 ━━━━━━━━━━━━━━━━━━━━ 3s 1s/step - loss: 0.0140 - val_loss: 0.0225
Epoch 129: early stopping
Restoring model weights from the end of the best epoch: 124.
```

In [111…

```python
def show_images(autoencoder, images=None):
    """
    Fonction pour afficher les images originales et leurs
    reconstructions générées par l'autoencodeur.

    Si aucun lot d'images n'est fourni, la fonction récupère un lot
    d'images depuis le générateur de données.

    Arguments :
    - autoencoder : L'autoencodeur entraîné, utilisé pour générer des
    reconstructions des images.
    - images (optionnel) : Un tableau d'images à afficher. Si non spécifié,
    un lot d'images est récupéré du générateur de données.

    Sortie :
    - Affichage des images originales et leurs reconstructions générées par
    l'autoencodeur.
    """

    if images is None:
        example_batch = next(data_flow)
        example_batch = example_batch[0]
        images = example_batch[:10]

    n_to_show = images.shape[0]

    # Génération des reconstructions des images
    reconst_images = autoencoder.predict(images)

    # Création de la figure pour l'affichage
    fig = plt.figure(figsize=(15, 3))
    fig.subplots_adjust(hspace=0.4, wspace=0.4)
```

```
    # Affichage des images originales
    for i in range(n_to_show):
        img = images[i].squeeze()
        sub = fig.add_subplot(2, n_to_show, i+1)
        sub.axis('off')
        sub.imshow(img)

    # Affichage des images reconstruites
    for i in range(n_to_show):
        img = reconst_images[i].squeeze()
        sub = fig.add_subplot(2, n_to_show, i+n_to_show+1)
        sub.axis('off')
        sub.imshow(img)
```

Visualisation des images reconstruits aprés avoir entrainé le modéle

In [112…

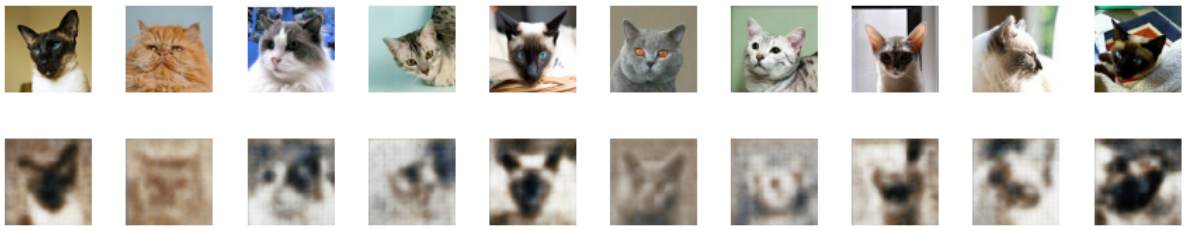
```
# Les deux fonctions suivantes ne nécessitent pas forcément d'être redéfinies
# Il s'agit plus d'une illustration de ce qu'il faut faire en cas de
# chargement indépendant

# Fonction de sampling utilisée dans la couche Lambda
@tf.keras.utils.register_keras_serializable()
def sampling(args):
    mean_mu, log_var = args
    epsilon = K.random_normal(shape=K.shape(mean_mu), mean=0., stddev=1.)
    return mean_mu + K.exp(log_var / 2) * epsilon

# Couche personnalisée pour le calcul de la perte KL
@tf.keras.utils.register_keras_serializable()
class KLLossLayer(tf.keras.layers.Layer):
    def call(self, inputs):
        mean_mu, log_var = inputs
        kl_loss = -0.5 * tf.reduce_sum(1 + log_var - tf.square(mean_mu) - tf.exp(l
        self.add_loss(kl_loss)
        return kl_loss

# Le modèle a besoin de connaître la définition de la total_loss
# Compilation du modèle avec la reconstruction loss (la KL est intégrée
# dans l'encodeur)
B = 1.0  # facteur de pondération de la reconstruction

@tf.keras.utils.register_keras_serializable()
def r_loss(y_true, y_pred):
    return K.mean(K.square(y_true - y_pred), axis=[1, 2, 3]) # MSE pixel à pixel

@tf.keras.utils.register_keras_serializable()
def total_loss(y_true, y_pred):
    return B * r_loss(y_true, y_pred) # KL-divergence ajoutée via .add_loss()
```

In [113…

```
# Chemin du modèle sauvegardé
model_name = "catGeneratingVAE.keras"
model_path = os.path.join(model_dir, model_name)

# Recharger le modèle avec les objets personnalisés nécessaires
vae = tf.keras.models.load_model(
    model_path,
    custom_objects={
        "KLLossLayer": KLLossLayer,
        "sampling": sampling,
        "r_loss": r_loss,
        "total_loss": total_loss
```

```
    }
)


show_images(vae)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 133ms/step
```

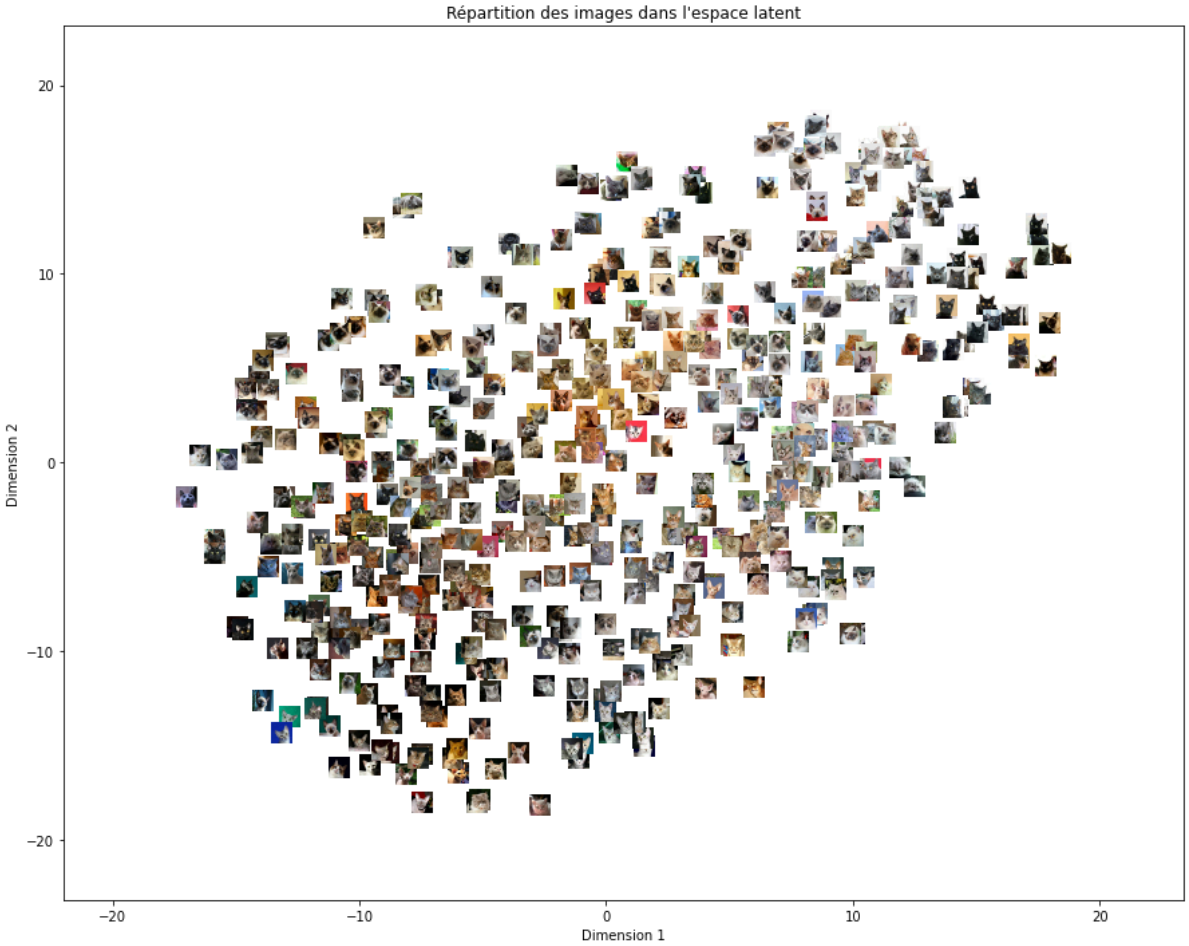


Visualisation de l'espace latent VAE

In [114…

```
# Chargement du variational autoencodeur
# Chemin du modèle sauvegardé
model_name = "catGeneratingVAE.keras"
# Chargement du modèle VAE avec les objets personnalisés
vae_model_path = os.path.join(model_dir, model_name)
vae = load_model(
    vae_model_path,
    custom_objects={
        "KLLossLayer": KLLossLayer,
        "sampling": sampling,
        "r_loss": r_loss,
        "total_loss": total_loss
    }
)

# Visualisation de l’espace latent du VAE
visualize_latent_space(
    encoder_model=vae_encoder,
    images=example_batch,
    num_images=batch_size,
    image_size=(32, 32),
    zoom=0.5
)
```

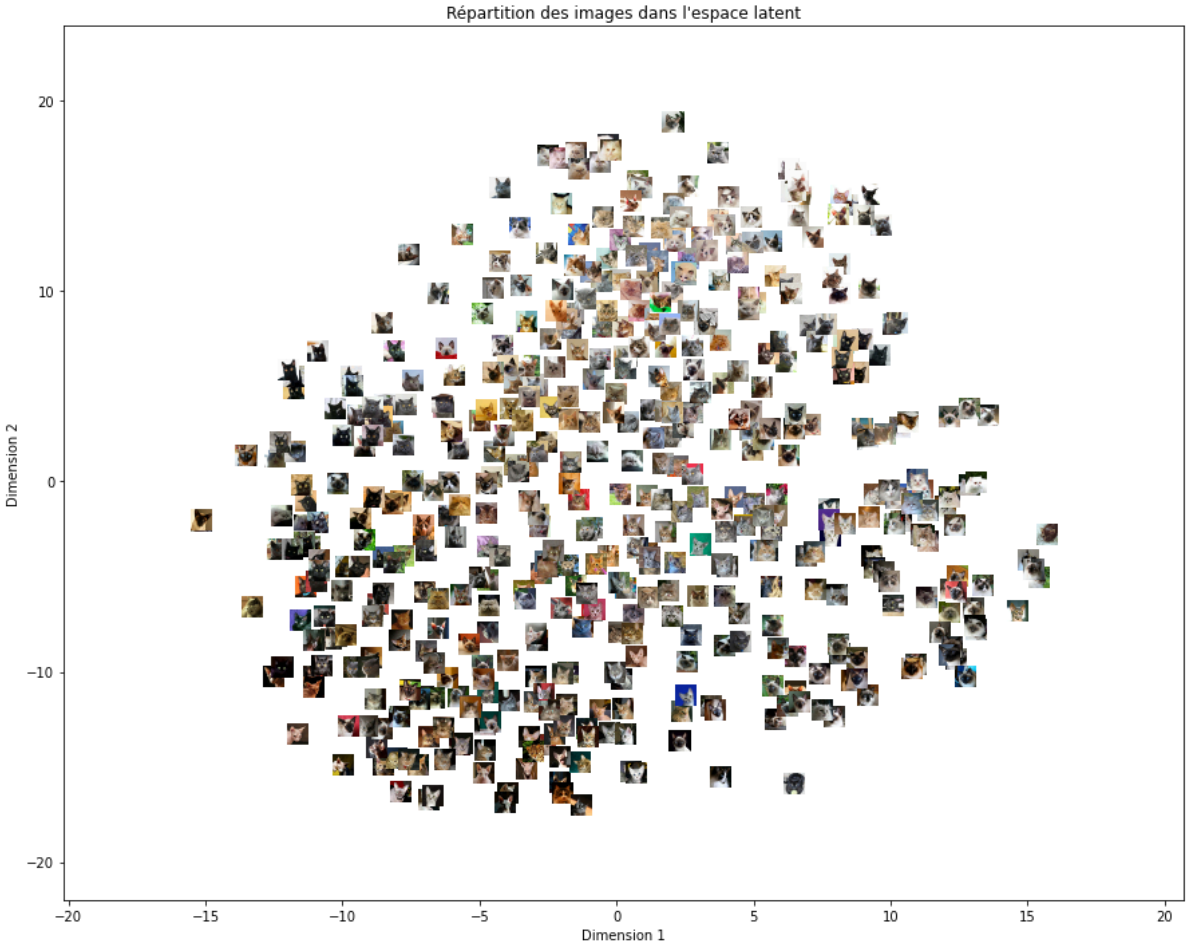
```
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
```



In [115…

```
#comparaison avec autoencodeur sans VAE:
# Chemin vers le modèle entraîné
model_path = os.path.join(model_dir, "catmodeltrain.keras")

# Chargement du modèle
autoencoder = load_model(model_path)

# Récupération du premier lot d'images
example_batch = next(data_flow)

# Extraction des images
example_batch = example_batch[0]
# Visualisation de l'espace latent avec les vignettes
visualize_latent_space(
    encoder_model=encoder,
    images=example_batch,
    num_images=batch_size,
    image_size=(32, 32),
    zoom=0.5
)
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
```



In [116…

```
##idk if theres much difference???
#celle de VAE semble un peu plus compacte?
```

Génération des images:

In [117…

```
# Chemin du modèle sauvegardé
model_name = "catGeneratingVAE.keras"
# Chargement du modèle VAE avec les objets personnalisés
vae_model_path = os.path.join(model_dir, model_name)
vae = load_model(
    vae_model_path,
    custom_objects={
        "KLLossLayer": KLLossLayer,
        "sampling": sampling,
        "r_loss": r_loss,
        "total_loss": total_loss
    }
)

_ = vae.predict(example_batch)  # passe des images pour activer le modèle

# Nombre d'images à générer
num_images = 10

# Tirage aléatoire de vecteurs latents depuis une normale centrée réduite
latent_vectors = np.random.normal(loc=0.0, scale=1, size=(num_images,
                                                          latent_dim))

vae_decoder = vae.get_layer('decoder')

# Génération des images à partir du décodeur du VAE
generated_images = vae_decoder.predict(latent_vectors)
```

```
# Affichage des images générées
plt.figure(figsize=(15, 3))
for i in range(num_images):
    ax = plt.subplot(1, num_images, i + 1)
    plt.imshow(generated_images[i])
    plt.axis("off")

plt.suptitle("Images générées aléatoirement à partir de l'espace latent (VAE)")
plt.tight_layout()
plt.show()
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 29ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step
```

In [118…

```
#c'est normal d'avoir ce résultat vu que le tirage des points était aléatoire
```

Ici on fait échantillonner les vecteurs latents qui se trouvent dans les zones denses pour produire un résultat plus réaliste

In [119…

```
# Sous-modèle pour obtenir les μ
encoder_mu_model = Model(inputs=vae.input,
                         outputs=vae.get_layer('mu').output)

# Obtenir les μ
mus = encoder_mu_model.predict(example_batch)

# Tirage aléatoire autour des μ
epsilon = np.random.normal(size=mus.shape)
scale = 1
latent_samples = mus + scale * epsilon

# Génération d'images à partir du décodeur
vae_decoder = vae.get_layer('decoder')
generated_images = vae_decoder.predict(latent_samples)

# Affichage
plt.figure(figsize=(20, 4))
for i in range(10):
    plt.subplot(1, 10, i + 1)
    plt.imshow(generated_images[i])
    plt.axis("off")
plt.suptitle("Échantillons latents proches des vrais μ")
plt.show()
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step
```

In [120…

```
#deja des résultats bcp mieux!
```

Plus loin avec les varations Autoencodeurs

In [121…

```
#Le nouveau VAE encodeur integrant les nouveaux param avec beta :
```

In [122…

```
# Facteur de pondération pour la divergence KL
beta = 3
#on l'a mis a 3 pour 'accentuer la structuration de l'espace latent'
```

In [123…

```
# VAE Encoder
def build_vae_encoder(input_dim, latent_dim, beta=1.0):
    """
    Construit l'encodeur d'un Variational Autoencoder (VAE) avec un facteur
    de pondération pour la perte KL.

    Parameters:
    - input_dim : tuple
        Dimension d'entrée des images (hauteur, largeur, canaux).
    - latent_dim : int
        Dimension de l'espace latent.
    - beta : float, optionnel
        Coefficient pour ajuster l'impact de la perte KL (par défaut 1.0).

    Returns:
    - tuple contenant les composants de l'encodeur.
    """
    tf.keras.backend.clear_session()

    # Définir l'entrée de l'encodeur
    vae_encoder_input = Input(shape=input_dim, name='vae_encoder_input')
    x = vae_encoder_input

    # Couches convolutives
    x = Conv2D(32, 3, strides=2, padding='same', name='vae_encoder_conv_1')(x)
    x = BatchNormalization()(x)
    x = LeakyReLU()(x)
    x = Conv2D(64, 3, strides=2, padding='same', name='vae_encoder_conv_2')(x)
    x = BatchNormalization()(x)
    x = LeakyReLU()(x)
    x = Conv2D(64, 3, strides=2, padding='same', name='vae_encoder_conv_3')(x)
    x = BatchNormalization()(x)
    x = LeakyReLU()(x)
    x = Conv2D(64, 3, strides=2, padding='same', name='vae_encoder_conv_4')(x)
    x = BatchNormalization()(x)
    x = LeakyReLU()(x)

    # Capture de la forme avant le Flatten
    shape_before_flattening = tf.keras.backend.int_shape(x)[1:]
    x = Flatten()(x)

    # Moyenne et variance log
    mean_mu = Dense(latent_dim, name='mu')(x)
    log_var = Dense(latent_dim, name='log_var')(x)

    # Fonction d'échantillonnage
    @register_keras_serializable(package='Custom', name='sampling')
    def sampling(args):
        mean_mu, log_var = args
        epsilon = K.random_normal(shape=K.shape(mean_mu), mean=0., stddev=1.)
```

```
        return mean_mu + K.exp(log_var / 2) * epsilon

    # Application du sampling
    vae_encoder_output = Lambda(sampling, output_shape=(latent_dim,),
                                name='vae_encoder_output')([mean_mu, log_var])

    # Couche de perte KL intégrée
    @register_keras_serializable(package='Custom', name='KLLossLayer')
    class KLLossLayer(tf.keras.layers.Layer):
        def __init__(self, beta=1.0, **kwargs):
            """
            Couche personnalisée pour la perte KL avec facteur de pondération.
            Paramètres:
            - beta : float, coefficient pour ajuster l'impact de la perte KL.
            """
            super(KLLossLayer, self).__init__(**kwargs)
            self.beta = beta

        def call(self, inputs):
            mean_mu, log_var = inputs
            # Calcul de la divergence KL entre la distribution latente
            # et une normale standard
            kl_loss = -0.5 * tf.reduce_sum(1 + log_var - tf.square(mean_mu) - tf.e
                                           axis=1)
            self.add_loss(self.beta * tf.reduce_mean(kl_loss))#Moyenne sur batch
            return kl_loss

    # Ajout de la couche KL à l'encodeur
    kl_loss_layer = KLLossLayer(beta=beta, name='kl_loss')([mean_mu, log_var])

    # Construction du modèle
    vae_encoder = Model(vae_encoder_input, vae_encoder_output,
                        name='vae_encoder')

    return vae_encoder_input, vae_encoder_output, mean_mu, log_var, shape_before_f
```

In [124…

```
from tensorflow.keras.utils import register_keras_serializable

# Création de l'encodeur VAE
vae_encoder_input, vae_encoder_output, mean_mu, log_var, shape_before_flattening,
    image_size,
    latent_dim,
    beta=beta
)

vae_encoder.summary()
```

Model: "vae_encoder"

Layer (type)	Output Shape	Param #	Connected to
vae_encoder_input (InputLayer)	(None, 128, 128, 3)	0	-
vae_encoder_conv_1 (Conv2D)	(None, 64, 64, 32)	896	vae_encoder_inpu…
batch_normalization (BatchNormalizatio…	(None, 64, 64, 32)	128	vae_encoder_conv…
leaky_re_lu (LeakyReLU)	(None, 64, 64, 32)	0	batch_normalizat…
vae_encoder_conv_2 (Conv2D)	(None, 32, 32, 64)	18,496	leaky_re_lu[0][0]
batch_normalizatio… (BatchNormalizatio…	(None, 32, 32, 64)	256	vae_encoder_conv…
leaky_re_lu_1 (LeakyReLU)	(None, 32, 32, 64)	0	batch_normalizat…
vae_encoder_conv_3 (Conv2D)	(None, 16, 16, 64)	36,928	leaky_re_lu_1[0]…
batch_normalizatio… (BatchNormalizatio…	(None, 16, 16, 64)	256	vae_encoder_conv…
leaky_re_lu_2 (LeakyReLU)	(None, 16, 16, 64)	0	batch_normalizat…
vae_encoder_conv_4 (Conv2D)	(None, 8, 8, 64)	36,928	leaky_re_lu_2[0]…
batch_normalizatio… (BatchNormalizatio…	(None, 8, 8, 64)	256	vae_encoder_conv…
leaky_re_lu_3 (LeakyReLU)	(None, 8, 8, 64)	0	batch_normalizat…
flatten (Flatten)	(None, 4096)	0	leaky_re_lu_3[0]…
mu (Dense)	(None, 200)	819,400	flatten[0][0]
log_var (Dense)	(None, 200)	819,400	flatten[0][0]
vae_encoder_output (Lambda)	(None, 200)	0	mu[0][0], log_var[0][0]

Total params: 1,732,944 (6.61 MB)

Trainable params: 1,732,496 (6.61 MB)

In [125…

```
#no huge changes in decoder
def build_vae_decoder(latent_dim, shape_before_flattening):
    """
    Construit le décodeur pour reconstruire une image à partir de
```

```
    l'espace latent.

    Parameters:
    - latent_dim : Dimension de l'espace latent.
    - shape_before_flattening : Dimensions avant Flatten
    (hauteur, largeur, canaux).

    Returns:
    - decoder_input : Input du modèle décodeur.
    - decoder_output : Sortie reconstruite.
    - decoder : Modèle complet du décodeur.
    """
    decoder_input = Input(shape=(latent_dim,), name='decoder_input')

    # Transformation dense vers une forme aplatie
    total_units = np.prod(shape_before_flattening)  # Produit des dimensions
    x = Dense(total_units, activation='relu',
              name='decoder_dense')(decoder_input)

    # Reshape pour correspondre à la représentation avant Flatten de l'encodeur
    x = Reshape(shape_before_flattening, name='decoder_reshape')(x)

    # Reconstruction de l'image avec des convolutions transposées
    x = Conv2DTranspose(64, kernel_size=3, strides=2, padding='same',
                        name='decoder_conv_1')(x)  # 7x7 en 14x14
    x = LeakyReLU(name='decoder_activation_1')(x)

    x = Conv2DTranspose(64, kernel_size=3, strides=2, padding='same',
                        name='decoder_conv_2')(x)  # 14x14 en 28x28
    x = LeakyReLU(name='decoder_activation_2')(x)

    x = Conv2DTranspose(32, kernel_size=3, strides=2, padding='same',
                        name='decoder_conv_3')(x)  # 28x28 en 56x56
    x = LeakyReLU(name='decoder_activation_3')(x)

    # Dernière couche pour ajuster la taille exacte à (100, 100, 3)
    x = Conv2DTranspose(3, kernel_size=3, strides=2, padding='same',
                        activation='sigmoid',
                        name='decoder_conv_4')(x)  # 56x56 en 112x112
    #x = Cropping2D(((6, 6), (6, 6)),
    #               name='decoder_cropping')(x)  # Ajuster à 100x100
    decoder_output = x

    return decoder_input, decoder_output, Model(decoder_input, decoder_output,
                                                name="vae_decoder")
```

In [126…

```
from tensorflow.keras.layers import Cropping2D
# Création du décodeur VAE
vae_decoder_input, vae_decoder_output, vae_decoder = build_vae_decoder(
    latent_dim,
    shape_before_flattening=shape_before_flattening
)

vae_decoder.summary()
```

Model: "vae_decoder"

Layer (type)	Output Shape	Param #
decoder_input (InputLayer)	(None, 200)	0
decoder_dense (Dense)	(None, 4096)	823,296
decoder_reshape (Reshape)	(None, 8, 8, 64)	0
decoder_conv_1 (Conv2DTranspose)	(None, 16, 16, 64)	36,928
decoder_activation_1 (LeakyReLU)	(None, 16, 16, 64)	0
decoder_conv_2 (Conv2DTranspose)	(None, 32, 32, 64)	36,928
decoder_activation_2 (LeakyReLU)	(None, 32, 32, 64)	0
decoder_conv_3 (Conv2DTranspose)	(None, 64, 64, 32)	18,464
decoder_activation_3 (LeakyReLU)	(None, 64, 64, 32)	0
decoder_conv_4 (Conv2DTranspose)	(None, 128, 128, 3)	867

Total params: 916,483 (3.50 MB)

Trainable params: 916,483 (3.50 MB)

In [127…

```
def create_vae():
    """
    Crée une nouvelle instance du VAE avec les mêmes paramètres que l'original.
    Returns:
    - vae : Model
        Une nouvelle instance du modèle VAE.
    """
    # Création de l'encodeur et du décodeur
    vae_encoder_input, vae_encoder_output, mean_mu, log_var, shape_before_flatteni
        input_dim=image_size,
        latent_dim=latent_dim,
        beta=beta
    )
    vae_decoder_input, vae_decoder_output, vae_decoder = build_vae_decoder(
        latent_dim=latent_dim,
        shape_before_flattening=shape_before_flattening
    )

    vae_autoencoder_input = vae_encoder_input
    vae_autoencoder_output = vae_decoder(vae_encoder_output)
    vae = Model(inputs=vae_autoencoder_input, outputs=vae_autoencoder_output,
                name="vae_autoencoder")

    # Compilation du modèle
    vae.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005),
```

```
                  loss="mse")
    return vae
```

In [128…

```
# Creation d'un vae pour vérifier que le modèle est bon
vae = create_vae()

vae.summary()
```

Model: "vae_autoencoder"

Layer (type)	Output Shape	Param #	Connected to
vae_encoder_input (InputLayer)	(None, 128, 128, 3)	0	-
vae_encoder_conv_1 (Conv2D)	(None, 64, 64, 32)	896	vae_encoder_inpu…
batch_normalization (BatchNormalizatio…	(None, 64, 64, 32)	128	vae_encoder_conv…
leaky_re_lu (LeakyReLU)	(None, 64, 64, 32)	0	batch_normalizat…
vae_encoder_conv_2 (Conv2D)	(None, 32, 32, 64)	18,496	leaky_re_lu[0][0]
batch_normalizatio… (BatchNormalizatio…	(None, 32, 32, 64)	256	vae_encoder_conv…
leaky_re_lu_1 (LeakyReLU)	(None, 32, 32, 64)	0	batch_normalizat…
vae_encoder_conv_3 (Conv2D)	(None, 16, 16, 64)	36,928	leaky_re_lu_1[0]…
batch_normalizatio… (BatchNormalizatio…	(None, 16, 16, 64)	256	vae_encoder_conv…
leaky_re_lu_2 (LeakyReLU)	(None, 16, 16, 64)	0	batch_normalizat…
vae_encoder_conv_4 (Conv2D)	(None, 8, 8, 64)	36,928	leaky_re_lu_2[0]…
batch_normalizatio… (BatchNormalizatio…	(None, 8, 8, 64)	256	vae_encoder_conv…
leaky_re_lu_3 (LeakyReLU)	(None, 8, 8, 64)	0	batch_normalizat…
flatten (Flatten)	(None, 4096)	0	leaky_re_lu_3[0]…
mu (Dense)	(None, 200)	819,400	flatten[0][0]
log_var (Dense)	(None, 200)	819,400	flatten[0][0]
vae_encoder_output (Lambda)	(None, 200)	0	mu[0][0], log_var[0][0]
vae_decoder (Functional)	(None, 128, 128, 3)	916,483	vae_encoder_outp…

Total params: 2,649,427 (10.11 MB)

Trainable params: 2,648,979 (10.11 MB)

In [129…

```python
def create_data_generator(directory, image_size, batch_size):
    """
    Crée un générateur.

    Args:
        directory (str): Chemin vers le répertoire contenant les images.
        image_size (tuple): Dimensions cibles des images (hauteur, largeur).
        batch_size (int): Taille des lots.

    Returns:
        data_gen (Iterator): Générateur Keras d'images normalisées.
        steps_per_epoch (int): Nombre de pas par époque.
    """
    generator = ImageDataGenerator(rescale=1./255)

    data_gen = generator.flow_from_directory(
    directory=src_dir,
    classes=['cat'],  # le sous-dossier contenant toutes les images
    target_size=image_size[:2],
    batch_size=batch_size,
    shuffle=True,
    seed=42,
    class_mode='input',
    subset='training'
    )

    return data_gen


def train_vae(model, data_gen, model_path, epochs=100, patience=10):
    """
    Entraîne un VAE sans validation avec callbacks intégrés.

    Args:
        model (Model): Modèle VAE compilé.
        data_gen (Iterator): Générateur Keras.
        steps_per_epoch (int): Nombre de pas par époque.
        model_path (str): Chemin pour sauvegarde du meilleur modèle.
        epochs (int): Nombre maximal d'époques.
        patience (int): Patience pour EarlyStopping.

    Returns:
        History: Historique d'entraînement.
    """
    callbacks = [
        EarlyStopping(monitor='loss', patience=patience, mode='min',
                      restore_best_weights=True, verbose=1),
        ModelCheckpoint(filepath=model_path, monitor='loss',
                        save_best_only=True, save_weights_only=False,
                        verbose=1, mode='min')
    ]

    history = model.fit(
        data_gen,
        epochs=epochs,
        callbacks=callbacks,
        verbose=1
    )

    return history
```

In [ ]:

In [ ]:

In [ ]:

In [130…
```
#setting image height and width i should change it according to my database
image_height=  128  # Hauteur des images
image_width = 128   # Largeur des images
# Pour les modèles
batch_size = 64     # Taille des lots
latent_dim = 16     # Dimension de l'espace latent pour le VAE
# Définition du répertoire cible
model_dir = "./Data_humanface_cat_without_caption_1000/"
```

In [131…
```
epochs = 120
##we only have one model -> cats we'll try on that
# Jeu de données
vae_cats = create_vae()
model_path = os.path.join(model_dir, "catGeneratingVAE.keras")
# Création du générateur
data_gen = create_data_generator(directory= "./Data_humanface_cat_without_caption_
                                 image_size=(image_height, image_width),
                                 batch_size=batch_size)
train_vae(vae_cats, data_gen, model_path, epochs)
```

```
Found 1000 images belonging to 1 classes.
Epoch 1/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 298ms/step - loss: 0.0730
Epoch 1: loss improved from None to 0.07339, saving model to ./Data_humanface_cat_
without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 8s 309ms/step - loss: 0.0734
Epoch 2/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 305ms/step - loss: 0.0714
Epoch 2: loss improved from 0.07339 to 0.06821, saving model to ./Data_humanface_c
at_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 5s 313ms/step - loss: 0.0682
Epoch 3/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 324ms/step - loss: 0.0618
Epoch 3: loss improved from 0.06821 to 0.06115, saving model to ./Data_humanface_c
at_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 5s 333ms/step - loss: 0.0612
Epoch 4/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 255ms/step - loss: 0.0549
Epoch 4: loss improved from 0.06115 to 0.05353, saving model to ./Data_humanface_c
at_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 260ms/step - loss: 0.0535
Epoch 5/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 263ms/step - loss: 0.0500
Epoch 5: loss improved from 0.05353 to 0.04870, saving model to ./Data_humanface_c
at_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 5s 270ms/step - loss: 0.0487
Epoch 6/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 190ms/step - loss: 0.0458
Epoch 6: loss improved from 0.04870 to 0.04499, saving model to ./Data_humanface_c
at_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 196ms/step - loss: 0.0450
Epoch 7/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 252ms/step - loss: 0.0414
Epoch 7: loss improved from 0.04499 to 0.03965, saving model to ./Data_humanface_c
at_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 259ms/step - loss: 0.0396
Epoch 8/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 248ms/step - loss: 0.0343
Epoch 8: loss improved from 0.03965 to 0.03310, saving model to ./Data_humanface_c
at_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 255ms/step - loss: 0.0331
Epoch 9/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 256ms/step - loss: 0.0307
Epoch 9: loss improved from 0.03310 to 0.02993, saving model to ./Data_humanface_c
at_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 262ms/step - loss: 0.0299
Epoch 10/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 233ms/step - loss: 0.0280
Epoch 10: loss improved from 0.02993 to 0.02778, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 238ms/step - loss: 0.0278
Epoch 11/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 240ms/step - loss: 0.0271
Epoch 11: loss improved from 0.02778 to 0.02670, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 246ms/step - loss: 0.0267
Epoch 12/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 244ms/step - loss: 0.0259
Epoch 12: loss improved from 0.02670 to 0.02588, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 251ms/step - loss: 0.0259
Epoch 13/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0255
Epoch 13: loss improved from 0.02588 to 0.02496, saving model to ./Data_humanface_
```

```
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 243ms/step - loss: 0.0250
Epoch 14/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 170ms/step - loss: 0.0239
Epoch 14: loss improved from 0.02496 to 0.02416, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 176ms/step - loss: 0.0242
Epoch 15/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 243ms/step - loss: 0.0236
Epoch 15: loss improved from 0.02416 to 0.02401, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 249ms/step - loss: 0.0240
Epoch 16/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 247ms/step - loss: 0.0237
Epoch 16: loss improved from 0.02401 to 0.02353, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 254ms/step - loss: 0.0235
Epoch 17/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0228
Epoch 17: loss improved from 0.02353 to 0.02286, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 244ms/step - loss: 0.0229
Epoch 18/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 274ms/step - loss: 0.0227
Epoch 18: loss improved from 0.02286 to 0.02256, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 282ms/step - loss: 0.0226
Epoch 19/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 265ms/step - loss: 0.0215
Epoch 19: loss improved from 0.02256 to 0.02218, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 271ms/step - loss: 0.0222
Epoch 20/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 246ms/step - loss: 0.0219
Epoch 20: loss improved from 0.02218 to 0.02210, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 253ms/step - loss: 0.0221
Epoch 21/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 253ms/step - loss: 0.0224
Epoch 21: loss improved from 0.02210 to 0.02207, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 263ms/step - loss: 0.0221
Epoch 22/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 215ms/step - loss: 0.0214
Epoch 22: loss improved from 0.02207 to 0.02203, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 222ms/step - loss: 0.0220
Epoch 23/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 254ms/step - loss: 0.0216
Epoch 23: loss improved from 0.02203 to 0.02164, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 261ms/step - loss: 0.0216
Epoch 24/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 238ms/step - loss: 0.0210
Epoch 24: loss improved from 0.02164 to 0.02126, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 245ms/step - loss: 0.0213
Epoch 25/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 241ms/step - loss: 0.0211
Epoch 25: loss improved from 0.02126 to 0.02120, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 248ms/step - loss: 0.0212
Epoch 26/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 235ms/step - loss: 0.0208
```

```
Epoch 26: loss improved from 0.02120 to 0.02101, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 243ms/step - loss: 0.0210
Epoch 27/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 236ms/step - loss: 0.0210
Epoch 27: loss improved from 0.02101 to 0.02085, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 243ms/step - loss: 0.0208
Epoch 28/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 234ms/step - loss: 0.0210
Epoch 28: loss did not improve from 0.02085
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 235ms/step - loss: 0.0213
Epoch 29/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0208
Epoch 29: loss improved from 0.02085 to 0.02071, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 243ms/step - loss: 0.0207
Epoch 30/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 245ms/step - loss: 0.0208
Epoch 30: loss improved from 0.02071 to 0.02050, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 251ms/step - loss: 0.0205
Epoch 31/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 241ms/step - loss: 0.0206
Epoch 31: loss improved from 0.02050 to 0.02045, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 247ms/step - loss: 0.0204
Epoch 32/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 242ms/step - loss: 0.0205
Epoch 32: loss did not improve from 0.02045
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 243ms/step - loss: 0.0205
Epoch 33/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 238ms/step - loss: 0.0201
Epoch 33: loss did not improve from 0.02045
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 239ms/step - loss: 0.0206
Epoch 34/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 246ms/step - loss: 0.0208
Epoch 34: loss did not improve from 0.02045
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 246ms/step - loss: 0.0207
Epoch 35/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 246ms/step - loss: 0.0199
Epoch 35: loss improved from 0.02045 to 0.02031, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 253ms/step - loss: 0.0203
Epoch 36/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 254ms/step - loss: 0.0200
Epoch 36: loss improved from 0.02031 to 0.02022, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 260ms/step - loss: 0.0202
Epoch 37/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 235ms/step - loss: 0.0196
Epoch 37: loss improved from 0.02022 to 0.01981, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 242ms/step - loss: 0.0198
Epoch 38/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 241ms/step - loss: 0.0203
Epoch 38: loss did not improve from 0.01981
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 242ms/step - loss: 0.0202
Epoch 39/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 243ms/step - loss: 0.0204
Epoch 39: loss did not improve from 0.01981
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 244ms/step - loss: 0.0199
Epoch 40/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 234ms/step - loss: 0.0197
```

```
Epoch 40: loss did not improve from 0.01981
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 235ms/step - loss: 0.0202
Epoch 41/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 234ms/step - loss: 0.0200
Epoch 41: loss did not improve from 0.01981
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 235ms/step - loss: 0.0202
Epoch 42/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 236ms/step - loss: 0.0193
Epoch 42: loss improved from 0.01981 to 0.01970, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 243ms/step - loss: 0.0197
Epoch 43/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 264ms/step - loss: 0.0197
Epoch 43: loss did not improve from 0.01970
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 264ms/step - loss: 0.0198
Epoch 44/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 241ms/step - loss: 0.0192
Epoch 44: loss improved from 0.01970 to 0.01955, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 247ms/step - loss: 0.0196
Epoch 45/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 248ms/step - loss: 0.0195
Epoch 45: loss improved from 0.01955 to 0.01939, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 254ms/step - loss: 0.0194
Epoch 46/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 182ms/step - loss: 0.0195
Epoch 46: loss did not improve from 0.01939
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 182ms/step - loss: 0.0197
Epoch 47/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 246ms/step - loss: 0.0190
Epoch 47: loss did not improve from 0.01939
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 246ms/step - loss: 0.0195
Epoch 48/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0194
Epoch 48: loss did not improve from 0.01939
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 238ms/step - loss: 0.0195
Epoch 49/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 236ms/step - loss: 0.0197
Epoch 49: loss did not improve from 0.01939
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 237ms/step - loss: 0.0198
Epoch 50/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0193
Epoch 50: loss improved from 0.01939 to 0.01919, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 244ms/step - loss: 0.0192
Epoch 51/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0189
Epoch 51: loss improved from 0.01919 to 0.01910, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 243ms/step - loss: 0.0191
Epoch 52/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 236ms/step - loss: 0.0188
Epoch 52: loss improved from 0.01910 to 0.01899, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 243ms/step - loss: 0.0190
Epoch 53/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0193
Epoch 53: loss did not improve from 0.01899
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 238ms/step - loss: 0.0192
Epoch 54/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 177ms/step - loss: 0.0184
Epoch 54: loss improved from 0.01899 to 0.01889, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 184ms/step - loss: 0.0189
Epoch 55/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 238ms/step - loss: 0.0192
Epoch 55: loss improved from 0.01889 to 0.01878, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 244ms/step - loss: 0.0188
Epoch 56/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 242ms/step - loss: 0.0187
Epoch 56: loss improved from 0.01878 to 0.01868, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 247ms/step - loss: 0.0187
Epoch 57/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0188
Epoch 57: loss did not improve from 0.01868
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 237ms/step - loss: 0.0187
Epoch 58/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 242ms/step - loss: 0.0184
Epoch 58: loss improved from 0.01868 to 0.01862, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 249ms/step - loss: 0.0186
Epoch 59/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 234ms/step - loss: 0.0184
Epoch 59: loss improved from 0.01862 to 0.01850, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 241ms/step - loss: 0.0185
Epoch 60/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 236ms/step - loss: 0.0181
Epoch 60: loss did not improve from 0.01850
16/16 ━━━━━━━━━━━━━━━━━━━━ 5s 236ms/step - loss: 0.0188
Epoch 61/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 236ms/step - loss: 0.0189
Epoch 61: loss did not improve from 0.01850
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 237ms/step - loss: 0.0187
Epoch 62/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 173ms/step - loss: 0.0184
Epoch 62: loss improved from 0.01850 to 0.01838, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 179ms/step - loss: 0.0184
Epoch 63/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 236ms/step - loss: 0.0179
Epoch 63: loss did not improve from 0.01838
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 236ms/step - loss: 0.0185
Epoch 64/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 233ms/step - loss: 0.0183
Epoch 64: loss did not improve from 0.01838
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 234ms/step - loss: 0.0184
Epoch 65/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0184
Epoch 65: loss improved from 0.01838 to 0.01832, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 245ms/step - loss: 0.0183
Epoch 66/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 236ms/step - loss: 0.0180
Epoch 66: loss improved from 0.01832 to 0.01820, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 242ms/step - loss: 0.0182
Epoch 67/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 238ms/step - loss: 0.0185
Epoch 67: loss did not improve from 0.01820
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 238ms/step - loss: 0.0186
Epoch 68/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 233ms/step - loss: 0.0187
Epoch 68: loss did not improve from 0.01820
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 234ms/step - loss: 0.0187
```

```
Epoch 69/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 235ms/step - loss: 0.0182
Epoch 69: loss did not improve from 0.01820
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 236ms/step - loss: 0.0182
Epoch 70/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 185ms/step - loss: 0.0186
Epoch 70: loss did not improve from 0.01820
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 186ms/step - loss: 0.0182
Epoch 71/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 265ms/step - loss: 0.0181
Epoch 71: loss did not improve from 0.01820
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 266ms/step - loss: 0.0184
Epoch 72/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 248ms/step - loss: 0.0189
Epoch 72: loss did not improve from 0.01820
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 249ms/step - loss: 0.0188
Epoch 73/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 245ms/step - loss: 0.0185
Epoch 73: loss improved from 0.01820 to 0.01809, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 252ms/step - loss: 0.0181
Epoch 74/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 265ms/step - loss: 0.0179
Epoch 74: loss improved from 0.01809 to 0.01786, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 274ms/step - loss: 0.0179
Epoch 75/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 275ms/step - loss: 0.0174
Epoch 75: loss improved from 0.01786 to 0.01777, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 5s 282ms/step - loss: 0.0178
Epoch 76/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 234ms/step - loss: 0.0173
Epoch 76: loss improved from 0.01777 to 0.01773, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 241ms/step - loss: 0.0177
Epoch 77/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 231ms/step - loss: 0.0178
Epoch 77: loss did not improve from 0.01773
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 233ms/step - loss: 0.0179
Epoch 78/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 170ms/step - loss: 0.0197
Epoch 78: loss did not improve from 0.01773
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 171ms/step - loss: 0.0191
Epoch 79/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 235ms/step - loss: 0.0179
Epoch 79: loss did not improve from 0.01773
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 236ms/step - loss: 0.0180
Epoch 80/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 242ms/step - loss: 0.0177
Epoch 80: loss improved from 0.01773 to 0.01770, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 249ms/step - loss: 0.0177
Epoch 81/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 252ms/step - loss: 0.0177
Epoch 81: loss did not improve from 0.01770
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 253ms/step - loss: 0.0177
Epoch 82/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 239ms/step - loss: 0.0172
Epoch 82: loss did not improve from 0.01770
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 240ms/step - loss: 0.0180
Epoch 83/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 236ms/step - loss: 0.0176
Epoch 83: loss did not improve from 0.01770
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 237ms/step - loss: 0.0179
Epoch 84/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 236ms/step - loss: 0.0176
Epoch 84: loss did not improve from 0.01770
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 236ms/step - loss: 0.0178
Epoch 85/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 232ms/step - loss: 0.0179
Epoch 85: loss did not improve from 0.01770
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 233ms/step - loss: 0.0180
Epoch 86/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 167ms/step - loss: 0.0171
Epoch 86: loss improved from 0.01770 to 0.01732, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 173ms/step - loss: 0.0173
Epoch 87/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 234ms/step - loss: 0.0175
Epoch 87: loss did not improve from 0.01732
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 234ms/step - loss: 0.0177
Epoch 88/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 253ms/step - loss: 0.0175
Epoch 88: loss did not improve from 0.01732
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 253ms/step - loss: 0.0176
Epoch 89/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 263ms/step - loss: 0.0174
Epoch 89: loss did not improve from 0.01732
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 264ms/step - loss: 0.0174
Epoch 90/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 245ms/step - loss: 0.0173
Epoch 90: loss improved from 0.01732 to 0.01716, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 252ms/step - loss: 0.0172
Epoch 91/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 234ms/step - loss: 0.0172
Epoch 91: loss did not improve from 0.01716
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 235ms/step - loss: 0.0174
Epoch 92/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 236ms/step - loss: 0.0170
Epoch 92: loss improved from 0.01716 to 0.01704, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 243ms/step - loss: 0.0170
Epoch 93/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 233ms/step - loss: 0.0172
Epoch 93: loss did not improve from 0.01704
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 233ms/step - loss: 0.0171
Epoch 94/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 171ms/step - loss: 0.0168
Epoch 94: loss improved from 0.01704 to 0.01675, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 177ms/step - loss: 0.0168
Epoch 95/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 233ms/step - loss: 0.0167
Epoch 95: loss did not improve from 0.01675
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 234ms/step - loss: 0.0168
Epoch 96/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 259ms/step - loss: 0.0164
Epoch 96: loss improved from 0.01675 to 0.01666, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 6s 264ms/step - loss: 0.0167
Epoch 97/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 235ms/step - loss: 0.0166
Epoch 97: loss improved from 0.01666 to 0.01664, saving model to ./Data_humanface_
cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 241ms/step - loss: 0.0166
Epoch 98/120
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 245ms/step - loss: 0.0170
Epoch 98: loss did not improve from 0.01664
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 245ms/step - loss: 0.0169
Epoch 99/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 238ms/step - loss: 0.0162
Epoch 99: loss did not improve from 0.01664
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 239ms/step - loss: 0.0167
Epoch 100/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0166
Epoch 100: loss improved from 0.01664 to 0.01662, saving model to ./Data_humanface
_cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 243ms/step - loss: 0.0166
Epoch 101/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 239ms/step - loss: 0.0169
Epoch 101: loss did not improve from 0.01662
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 240ms/step - loss: 0.0172
Epoch 102/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 173ms/step - loss: 0.0170
Epoch 102: loss did not improve from 0.01662
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 174ms/step - loss: 0.0167
Epoch 103/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0167
Epoch 103: loss did not improve from 0.01662
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 237ms/step - loss: 0.0168
Epoch 104/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 240ms/step - loss: 0.0170
Epoch 104: loss did not improve from 0.01662
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 241ms/step - loss: 0.0169
Epoch 105/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 236ms/step - loss: 0.0168
Epoch 105: loss did not improve from 0.01662
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 237ms/step - loss: 0.0169
Epoch 106/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 245ms/step - loss: 0.0168
Epoch 106: loss improved from 0.01662 to 0.01657, saving model to ./Data_humanface
_cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 252ms/step - loss: 0.0166
Epoch 107/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 246ms/step - loss: 0.0162
Epoch 107: loss improved from 0.01657 to 0.01646, saving model to ./Data_humanface
_cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 253ms/step - loss: 0.0165
Epoch 108/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0165
Epoch 108: loss did not improve from 0.01646
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 238ms/step - loss: 0.0165
Epoch 109/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 247ms/step - loss: 0.0163
Epoch 109: loss improved from 0.01646 to 0.01619, saving model to ./Data_humanface
_cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 253ms/step - loss: 0.0162
Epoch 110/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 172ms/step - loss: 0.0164
Epoch 110: loss did not improve from 0.01619
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 173ms/step - loss: 0.0163
Epoch 111/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 252ms/step - loss: 0.0163
Epoch 111: loss did not improve from 0.01619
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 253ms/step - loss: 0.0164
Epoch 112/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 237ms/step - loss: 0.0165
Epoch 112: loss did not improve from 0.01619
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 238ms/step - loss: 0.0167
Epoch 113/120
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 251ms/step - loss: 0.0164
Epoch 113: loss did not improve from 0.01619
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 251ms/step - loss: 0.0165
Epoch 114/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 241ms/step - loss: 0.0165
Epoch 114: loss improved from 0.01619 to 0.01603, saving model to ./Data_humanface
_cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 249ms/step - loss: 0.0160
Epoch 115/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 238ms/step - loss: 0.0163
Epoch 115: loss did not improve from 0.01603
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 238ms/step - loss: 0.0161
Epoch 116/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 244ms/step - loss: 0.0159
Epoch 116: loss improved from 0.01603 to 0.01591, saving model to ./Data_humanface
_cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 251ms/step - loss: 0.0159
Epoch 117/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 234ms/step - loss: 0.0161
Epoch 117: loss did not improve from 0.01591
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 235ms/step - loss: 0.0161
Epoch 118/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 168ms/step - loss: 0.0158
Epoch 118: loss improved from 0.01591 to 0.01588, saving model to ./Data_humanface
_cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 174ms/step - loss: 0.0159
Epoch 119/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 235ms/step - loss: 0.0157
Epoch 119: loss did not improve from 0.01588
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 236ms/step - loss: 0.0160
Epoch 120/120
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 246ms/step - loss: 0.0152
Epoch 120: loss improved from 0.01588 to 0.01549, saving model to ./Data_humanface
_cat_without_caption_1000/catGeneratingVAE.keras
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 251ms/step - loss: 0.0155
Restoring model weights from the end of the best epoch: 120.
<keras.src.callbacks.history.History at 0x714661717760>
```

Out[131]:

import os for root, dirs, files in os.walk("./Data_humanface_cat_without_caption_1000/images"): level = root.replace("./Data_humanface_cat_without_caption_1000/images", "").count(os.sep) indent = " " * level print(f"{indent}{os.path.basename(root)}/") if files: print(f"{indent} → {len(files)} files, e.g. {files[0]}")

In [132…

```python
# Rappel de la fonction et de la classe pour connaitre le code
@register_keras_serializable(package='Custom', name='sampling')
def sampling(args):
    mean_mu, log_var = args
    epsilon = K.random_normal(shape=K.shape(mean_mu), mean=0., stddev=1.)
    return mean_mu + K.exp(log_var / 2) * epsilon

@register_keras_serializable(package='Custom', name='KLLossLayer')
class KLLossLayer(tf.keras.layers.Layer):
    def __init__(self, beta=1.0, **kwargs):
        super(KLLossLayer, self).__init__(**kwargs)
        self.beta = beta

    def call(self, inputs):
        mean_mu, log_var = inputs
        kl_loss = -0.5 * tf.reduce_sum(
            1 + log_var - tf.square(mean_mu) - tf.exp(log_var),
            axis=1
        )
        self.add_loss(self.beta * tf.reduce_mean(kl_loss))
        return kl_loss
```

In [133…

```python
model_path = os.path.join(model_dir, "catGeneratingVAE.keras")
vae_cats = load_model(
    model_path,
    custom_objects={
        "sampling": sampling,
        "KLLossLayer": KLLossLayer
    }
)
print("Modèle cats chargé.")
```

```
Modèle cats chargé.
```

In [134…

```python
#pour voir les modéles reconstruits:
```

In [135…

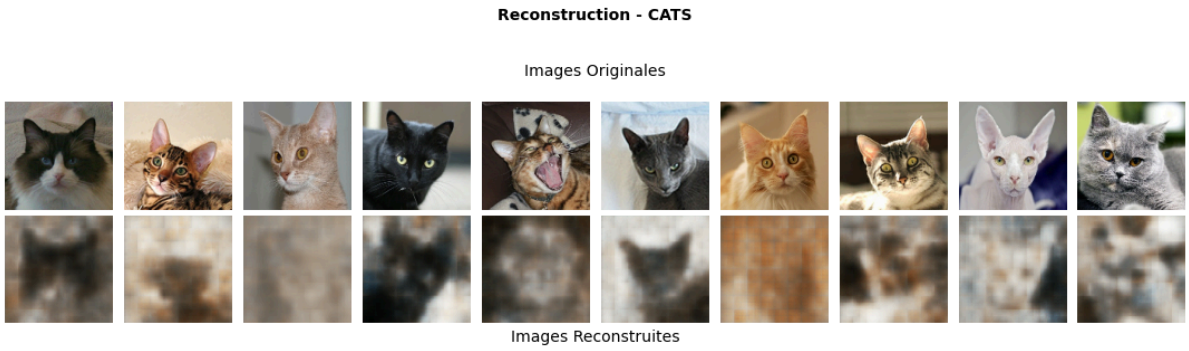
```python
def plot_vae_reconstructions(vae_model, data_gen, title=None):
    """
    Affiche 10 images originales et leurs reconstructions par le VAE.

    Args:
        vae_model : modèle VAE chargé
        data_gen  : générateur de données (ImageDataGenerator)
        title     : titre facultatif pour le graphique
    """
    x_batch = next(data_gen)
    if isinstance(x_batch, tuple):
        x_batch = x_batch[0]

    x_batch = x_batch[:10]
    reconstructions = vae_model.predict(x_batch, verbose=0)

    fig = plt.figure(figsize=(15, 5))

    for i in range(10):
        ax = plt.subplot(2, 10, i + 1)
        plt.imshow(np.clip(x_batch[i], 0, 1))
        plt.axis("off")

        ax = plt.subplot(2, 10, i + 11)
```

```
        plt.imshow(np.clip(reconstructions[i], 0, 1))
        plt.axis("off")

    if title:
        fig.suptitle(title, fontsize=14, fontweight='bold', y=0.93)

    fig.text(0.5, 0.76, "Images Originales", fontsize=14, ha='center')
    fig.text(0.5, 0.10, "Images Reconstruites", fontsize=14, ha='center')

    plt.tight_layout(rect=[0, 0.05, 1, 0.86])
    plt.show()
```

In [136…

```
data_gen_cats_seules = create_data_generator(
    directory="./Data_humanface_cat_without_caption_1000/images",
    image_size=(image_height, image_width),
    batch_size=batch_size
)
plot_vae_reconstructions(vae_cats, data_gen_cats_seules,
                         title="Reconstruction - CATS")
```

```
Found 1000 images belonging to 1 classes.
```



In [137…

```
#still blurry i hope next steps will fix that
```

In [138…

```
#ici on va visualiser plus de détails sur l'espace latente sur notre exemple pour
```

In [139…

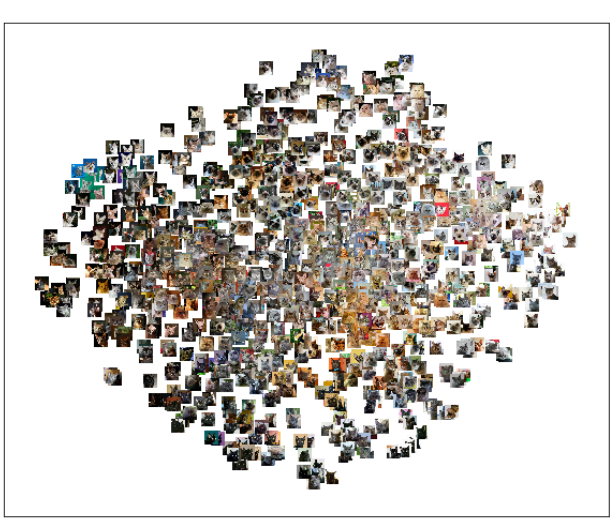
```
def plot_latent_space_with_density(
    vae_model,
    model_dir,
    num_samples=500,
    grid_size=100,
    image_size=(128, 128),
    zoom=0.1
):
    """
    Affiche côte à côte :
    - à gauche : le t-SNE avec vignettes d'images
    - à droite : la densité estimée ("montagnes") dans l'espace latent

    Args:
        vae_model     : modèle VAE (avec la couche "vae_encoder_output")
        dataset_name  : nom du dataset (clé dans dataset_paths)
        dataset_paths : dict des chemins des datasets
        model_dir     : répertoire des fichiers .npz
        num_samples   : nombre d'images à utiliser
        grid_size     : résolution de la grille KDE
        image_size    : taille de chaque vignette (pixels)
        zoom          : facteur de zoom pour les vignettes
```

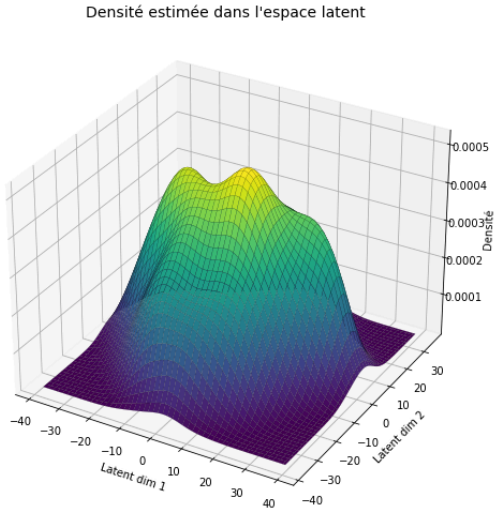
```
    """
    #dataset_dir = dataset_paths[dataset_name]
    valid_ext = (".jpg", ".jpeg", ".png", ".bmp", ".webp")

    image_paths = []
    for root, _, files in os.walk(model_dir):
        for f in files:
            if f.lower().endswith(valid_ext):
                image_paths.append(os.path.join(root, f))

    image_paths = image_paths[:num_samples]

    if len(image_paths) == 0:
        raise ValueError("No images found. Check dataset path.")

#    images = [
#        np.array(Image.open(os.path.join(model_dir, f))) / 255.0
#        for f in filenames
#    ]
    images = []
    for p in image_paths:
        img = Image.open(p).convert("RGB").resize((image_size[0], image_size[1]))
        img = np.array(img) / 255.0
        images.append(img)

    x = np.stack(images)


    encoder_output = vae_model.get_layer("vae_encoder_output").output
    encoder = Model(inputs=vae_model.input, outputs=encoder_output)
    z = encoder.predict(x, verbose=0)
    z_tsne = TSNE(n_components=2, random_state=42).fit_transform(z)

    fig = plt.figure(figsize=(16, 7))

    # --- Subplot gauche : t-SNE avec vignettes ---
    ax1 = fig.add_subplot(1, 2, 1)
    #ax1.set_title(f"t-SNE avec vignettes — {}, fontsize=14)

    for i, (x_tsne, y_tsne) in enumerate(z_tsne):
        thumb = array_to_img(x[i])
        thumb = thumb.resize(image_size)
        imagebox = OffsetImage(thumb, zoom=zoom)
        ab = AnnotationBbox(imagebox, (x_tsne, y_tsne), frameon=False)
        ax1.add_artist(ab)

    ax1.set_xlim(z_tsne[:, 0].min() - 5, z_tsne[:, 0].max() + 5)
    ax1.set_ylim(z_tsne[:, 1].min() - 5, z_tsne[:, 1].max() + 5)
    ax1.set_xticks([])
    ax1.set_yticks([])
    ax1.grid(False)

    # --- Subplot droit : densité KDE 3D ---
    kde = gaussian_kde(z_tsne.T)
    x_vals = np.linspace(z_tsne[:, 0].min() - 5, z_tsne[:, 0].max() + 5,
                         grid_size)
    y_vals = np.linspace(z_tsne[:, 1].min() - 5, z_tsne[:, 1].max() + 5,
                         grid_size)
    X, Y = np.meshgrid(x_vals, y_vals)
    Z = kde(np.vstack([X.ravel(), Y.ravel()])).reshape(X.shape)

    ax2 = fig.add_subplot(1, 2, 2, projection='3d')
    ax2.plot_surface(X, Y, Z, cmap=cm.viridis, edgecolor='k', linewidth=0.1)
    ax2.set_title("Densité estimée dans l'espace latent", fontsize=14)
```

```
    ax2.set_xlabel("Latent dim 1")
    ax2.set_ylabel("Latent dim 2")
    ax2.set_zlabel("Densité")

    plt.tight_layout()
    plt.show()
```

In [140…

```
from scipy.stats import gaussian_kde
import matplotlib.cm as cm
model_dir= "./Data_humanface_cat_without_caption_1000/images"
plot_latent_space_with_density(
    vae_model=vae_cats,
    model_dir=model_dir,
    num_samples=1000
)
```





#okay interesting #maintenant on defini la fonction de l'histogramme representant les dimensions latentes def plot_latent_histogram_from_vae(vae_model, model_dir, title="Histogramme des vecteurs latents", dimensions=8, bins=50): """ Affiche un histogramme groupé des valeurs latentes pour chaque catégorie. Args: vae_model : modèle VAE complet dataset_name : nom du dataset dans dataset_paths dataset_paths : dict {nom: chemin} model_dir : chemin vers les fichiers .npz title : titre du graphique dimensions : nombre de dimensions latentes à inclure (par défaut: 8) bins : nombre de bins dans l'histogramme """ valid_ext = (".jpg", ".jpeg", ".png", ".bmp", ".webp") image_paths = [] for root, _, files in os.walk(model_dir): for f in files: if f.lower().endswith(valid_ext): image_paths.append(os.path.join(root, f)) if len(image_paths) == 0: raise ValueError("No images found.") images = [] for p in image_paths: img = Image.open(p).convert("RGB") img = img.resize(image_size[:2]) #so it can be 2d not 3d we dont need? the rgb in this step img = np.array(img) / 255.0 images.append(img) x = np.stack(images) encoder_output = vae_model.get_layer("vae_encoder_output").output encoder = Model(inputs=vae_model.input, outputs=encoder_output) latent_vectors = encoder.predict(x, batch_size=8, verbose=0) #j'ai ajouté batchsize pour que ça soit moins d'étapes et noyau #ne plante pas dims_to_plot = range(min(dimensions, latent_vectors.shape[1])) #flatten latten values?? latent_values = latent_vectors[:, :dimensions].flatten() latent_values = np.asarray(latent_values).ravel() #added bc issue with plotting plt.figure(figsize=(12, 6)) sns.histplot(x=latent_values, bins=bins, kde=True, stat="density") plt.title(title, fontsize=16) plt.xlabel("Valeurs latentes") plt.ylabel("Densité") plt.grid(alpha=0.3) plt.show()import seaborn as sns model_dir= "./Data_humanface_cat_without_caption_1000/images" plot_latent_histogram_from_vae(vae_model=vae_cats, model_dir=model_dir, dimensions=8 #le nb de dimensions latentes?)

In [ ]:

Maintenant on veut visualiser pour chaque dimension l'estimation de densité

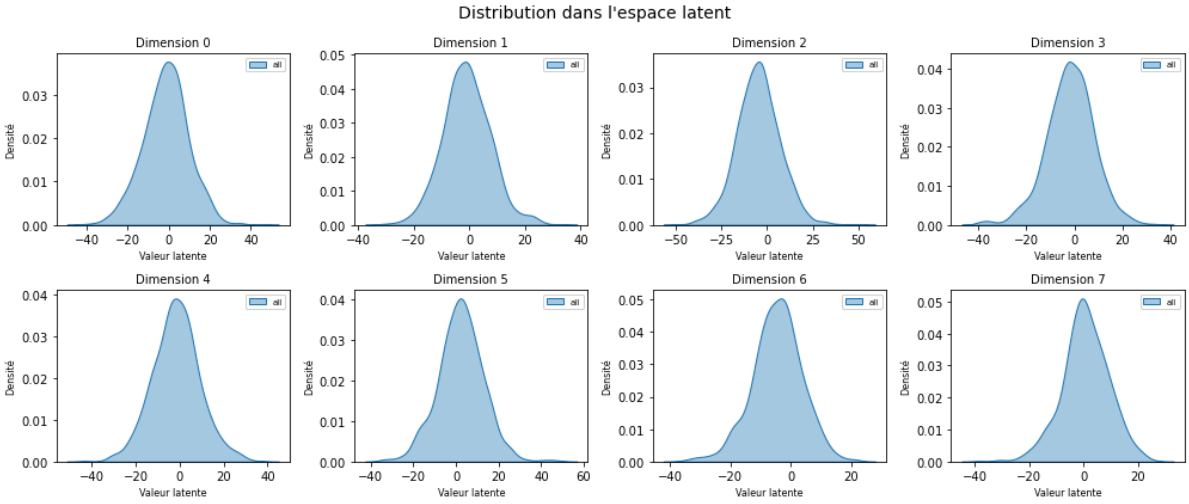
In [141…

```
def plot_latent_distribution_from_vae(
    vae_model,
```

```
    model_dir,
    title="Distribution dans l'espace latent",
    dimensions=8,
    image_size=(128, 128)
):
    """
    Visualise la distribution KDE des dimensions latentes par sous-dossier (catégo
    Les catégories sont inférées automatiquement depuis les sous-dossiers de model
    Si tout est dans un seul dossier, une seule courbe est affichée.

    Args:
        vae_model  : modèle VAE complet (avec couche "vae_encoder_output")
        model_dir  : dossier racine contenant les images (avec ou sans sous-dossie
        title      : titre du graphique
        dimensions : nombre de dimensions latentes à tracer
        image_size : taille de redimensionnement des images (H, W)
    """
    valid_ext = (".jpg", ".jpeg", ".png", ".bmp", ".webp")


    category_paths = {}
    for root, dirs, files in os.walk(model_dir):
        imgs = [os.path.join(root, f) for f in files if f.lower().endswith(valid_e
        if imgs:
            cat = os.path.relpath(root, model_dir)
            cat = "all" if cat == "." else cat
            category_paths.setdefault(cat, []).extend(imgs)

    if not category_paths:
        raise ValueError(f"No images found in {model_dir}")

    encoder_output = vae_model.get_layer("vae_encoder_output").output
    encoder = Model(inputs=vae_model.input, outputs=encoder_output)

    category_latents = {}
    for cat, paths in category_paths.items():
        imgs = []
        for p in paths:
            img = Image.open(p).convert("RGB").resize((image_size[1], image_size[0
            imgs.append(np.array(img) / 255.0)
        x = np.stack(imgs)
        category_latents[cat] = encoder.predict(x, batch_size=16, verbose=0)

    num_latent_dims = next(iter(category_latents.values())).shape[1]
    dims_to_plot = range(min(dimensions, num_latent_dims))
    n_cols = int(np.ceil(len(dims_to_plot) / 2))

    plt.figure(figsize=(14, 6))
    for i, dim in enumerate(dims_to_plot):
        plt.subplot(2, n_cols, i + 1)
        for cat, latents in category_latents.items():
            class_data = latents[:, dim]
            if len(class_data) > 1:
                sns.kdeplot(class_data, label=cat, fill=True, alpha=0.4)
        plt.title(f"Dimension {dim}", fontsize=10)
        plt.xlabel("Valeur latente", fontsize=8)
        plt.ylabel("Densité", fontsize=8)
        plt.legend(fontsize=7)

    plt.suptitle(title, fontsize=14)
    plt.tight_layout()
    plt.show()
```

In [142…

```
import seaborn as sns
model_dir= "./Data_humanface_cat_without_caption_1000/images/cat" #ici j'ai precis
#de plusieurs categories qui sont sur le même chemin
plot_latent_distribution_from_vae(
    vae_model=vae_cats,
    model_dir=model_dir,
    dimensions=8 #le nb de dimensions latentes?
)
```



NOUVELLE STRATEGIE DE CLASSIFICATION: Génération par classe

In [143…

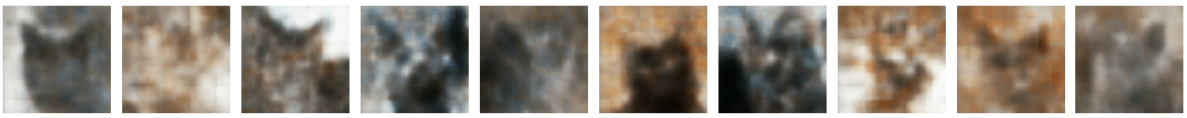
```
def generate_images_by_class_from_images(
    model,
    model_dir,
    category_name,
    num_samples=10,
    noise_scale=0.0,
    image_size=(128, 128)
):
    """
    Génère et affiche des images depuis le VAE à partir d'images d'une classe
    donnée, inférée depuis les sous-dossiers de model_dir.

    Args:
        model          : modèle VAE complet (avec couche "vae_encoder_output"
                         et "vae_decoder")
        model_dir      : dossier racine contenant les images, organisé en
                         sous-dossiers par catégorie (ex: model_dir/cats/)
                         OU dossier plat si une seule catégorie
        category_name  : nom de la catégorie = nom du sous-dossier
                         (ex: "cats", "humans"). Si model_dir est plat,
                         utilisez "all".
        num_samples    : nombre d'images à générer
        noise_scale    : écart-type du bruit latent ajouté
        image_size     : taille de redimensionnement (H, W)
    """
    valid_ext = (".jpg", ".jpeg", ".png", ".bmp", ".webp")

    # --- Collecte des chemins selon la catégorie ---
    # Build a {category: [paths]} map from subfolders, same as other functions
    category_paths = {}
    for root, _, files in os.walk(model_dir):
```

```
        imgs = [os.path.join(root, f) for f in files if f.lower().endswith(valid_e
        if imgs:
            cat = os.path.relpath(root, model_dir)
            cat = "all" if cat == "." else cat
            category_paths.setdefault(cat, []).extend(imgs)

    if not category_paths:
        raise ValueError(f"Aucune image trouvée dans '{model_dir}'.")

    if category_name not in category_paths:
        available = list(category_paths.keys())
        raise ValueError(
            f"Catégorie '{category_name}' introuvable.\n"
            f"Catégories disponibles : {available}"
        )

    image_paths = category_paths[category_name]

    images = []
    for p in image_paths:
        img = Image.open(p).convert("RGB").resize((image_size[1], image_size[0]))
        images.append(np.array(img) / 255.0)

    x = np.stack(images)

    encoder_output = model.get_layer("vae_encoder_output").output
    encoder = Model(inputs=model.input, outputs=encoder_output)
    z_all = encoder.predict(x, batch_size=16, verbose=0)

    decoder = model.get_layer("vae_decoder")

    plt.figure(figsize=(num_samples * 1.5, 2.5))
    for i in range(num_samples):
        idx = np.random.randint(len(z_all))
        noisy_z = z_all[idx] + np.random.normal(0, noise_scale, size=z_all.shape[1
        decoded = decoder.predict(
            np.expand_dims(noisy_z, axis=0), verbose=0
        )[0]
        ax = plt.subplot(1, num_samples, i + 1)
        plt.imshow(np.clip(decoded, 0, 1))
        plt.axis("off")

    plt.suptitle(
        f"Génération par classe — {category_name} "
        f"({len(image_paths)} images source)",
        fontsize=13,
        fontweight="bold"
    )
    plt.tight_layout()
    plt.show()
```

In [144…

```
model_dir= "./Data_humanface_cat_without_caption_1000/images/"

generate_images_by_class_from_images(
    model=vae_cats,
    model_dir=model_dir,
    category_name="cat",
    num_samples=10
)
```

Génération par classe — cat (1000 images source)



In [ ]: